

- [Kafka — что это и зачем](#)
 - [Проблема интеграций и decoupling](#)
 - [Apache Kafka — определение и применение](#)
- [Модель данных — топики, партиции, offsets](#)
 - [Topic](#)
 - [Partition и offset](#)
 - [Анатомия сообщения](#)
 - [Сериализация и десериализация](#)
- [Кластер — брокеры, репликация, координация](#)
 - [Брокеры и кластер](#)
 - [Репликация — replication factor, leader, ISR](#)
 - [acks и durability — гарантии записи](#)
 - [Zookeeper и KRaft — кто управляет кластером](#)
- [Окружение — установка, CLI, Java-проект](#)
 - [Установка Kafka](#)
 - [Kafka CLI — как вызывать](#)
 - [kafka-topics.sh — управление топиками](#)
 - [Kafka SDK и Java-проект](#)
- [Producer](#)
 - [Как producer пишет данные — партиции и ключи](#)
 - [kafka-console-producer.sh](#)
 - [Java Producer — базовый](#)
 - [Callback и sticky partitioner](#)
 - [Producer с ключами](#)
- [Consumer](#)
 - [Как consumer читает данные](#)
 - [kafka-console-consumer.sh](#)
 - [Java Consumer — базовый](#)
 - [Graceful Shutdown](#)
- [Consumer Groups и rebalance](#)
 - [Consumer group — как группа делит топик](#)
 - [Группы на практике — CLI и Java](#)
 - [Rebalance — eager, cooperative, static membership](#)
- [Offsets и семантики доставки](#)
 - [Consumer offsets — где группа остановилась](#)

- [Коммит offsets — авто и ручной](#)
 - [Сброс offsets — reset-offsets](#)
 - [Delivery semantics — at most, at least, exactly once](#)
 - [Итоговая схема](#)
 - [Вопросы к собеседованию](#)
 - [Общее и архитектура](#)
 - [Топики, партиции и хранение](#)
 - [Брокеры и репликация](#)
 - [Producer](#)
 - [Consumer и группы](#)
 - [Offsets и гарантии доставки](#)
 - [API и экосистема](#)
 - [Сравнение с другими системами](#)
-

Kafka — что это и зачем

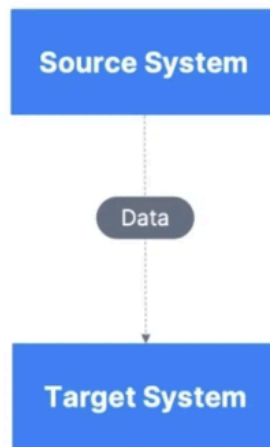
Проблема интеграций и decoupling

Одни системы данные **производят** (source system), другие **потребляют** (target system). Пока источник и цель одни — хватает одной прямой интеграции. С ростом компании связи «каждый с каждым» превращаются в паутину: **4 источника × 6 целей = 24 интеграции**.

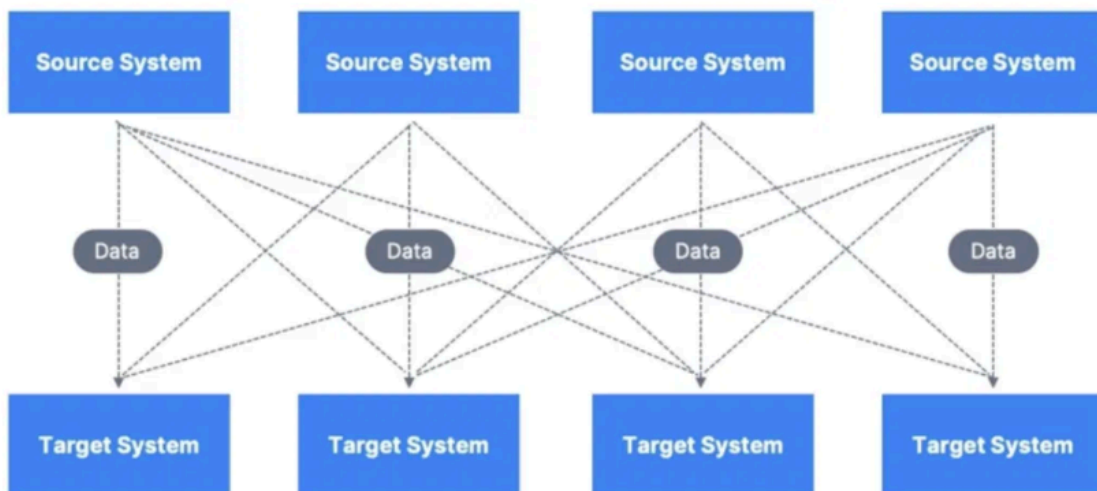
Каждая интеграция — это:

- **протокол** — TCP, HTTP, REST, FTP, JDBC...
- **формат данных** — Binary, CSV, JSON, Avro, Protobuf...
- **схема и её эволюция** — какой формы данные и как она меняется;
- **нагрузка на источник** — к нему подключается каждый потребитель.

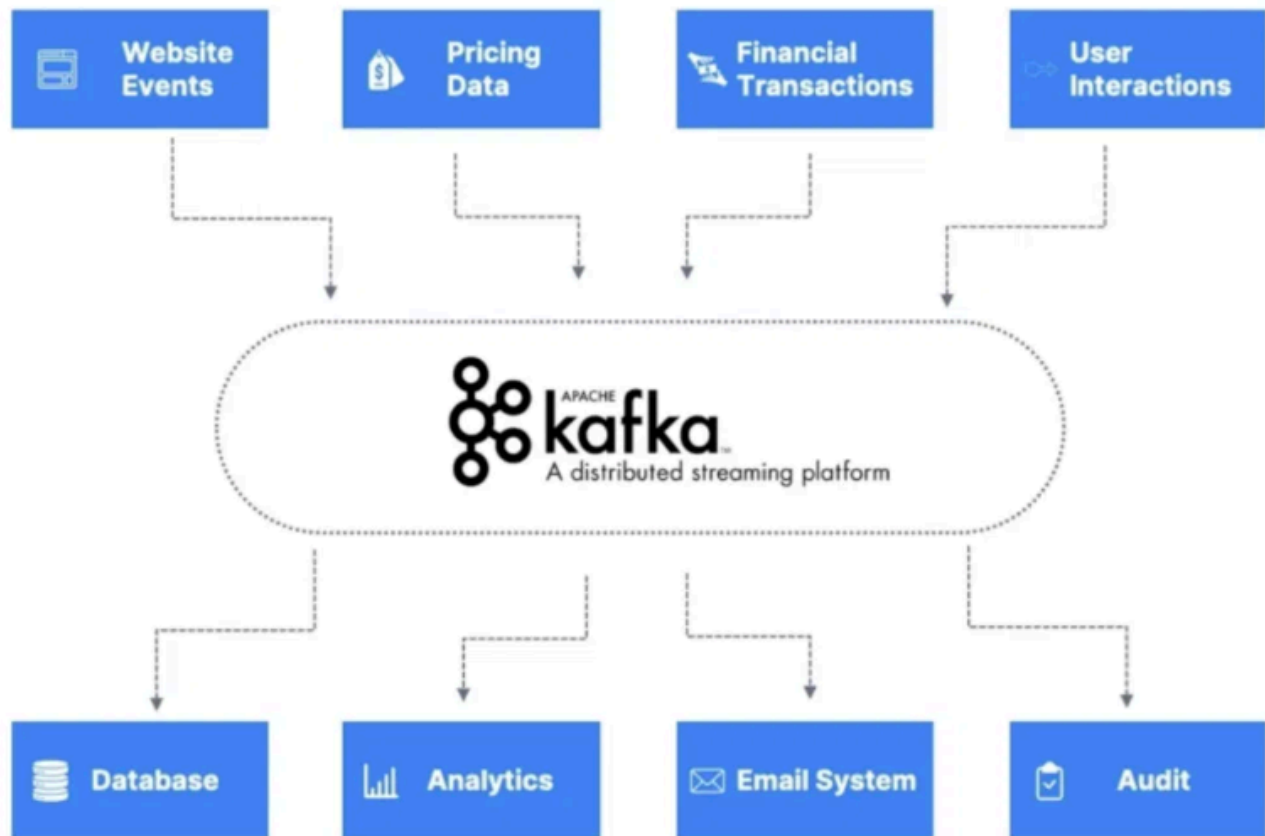
simple



complicated



Решение: между источниками и потребителями ставится общий посредник. Источники пишут в него, потребители читают из него и друг о друге не знают — это **decoupling** (развязывание) систем. Новый потребитель подключается только к посреднику, источник не трогается.



Apache Kafka — определение и применение

Определение: Apache Kafka — распределённая платформа потоковой передачи данных (distributed streaming platform). Источники пишут в неё сообщения, потребители читают их независимо друг от друга.

Свойство	Что значит
Происхождение	создана в LinkedIn, сейчас open-source проект Apache; основные контрибьюторы — Confluent, IBM, Cloudera
Отказоустойчивость	распределённая архитектура, fault tolerant
Горизонтальное масштабирование	сотни брокеров, миллионы сообщений в секунду
Производительность	задержка < 10 мс — real time
Распространённость	2000+ компаний, 80% Fortune 100 (Airbnb, LinkedIn, Uber, Netflix, Walmart)

Где применяется:

- messaging system и pub/sub между микросервисами;
- activity tracking, сбор метрик и логов;
- потоковая обработка (Kafka Streams);
- decoupling систем;
- интеграция с Big Data: Spark, Flink, Storm, Hadoop.

Примеры:

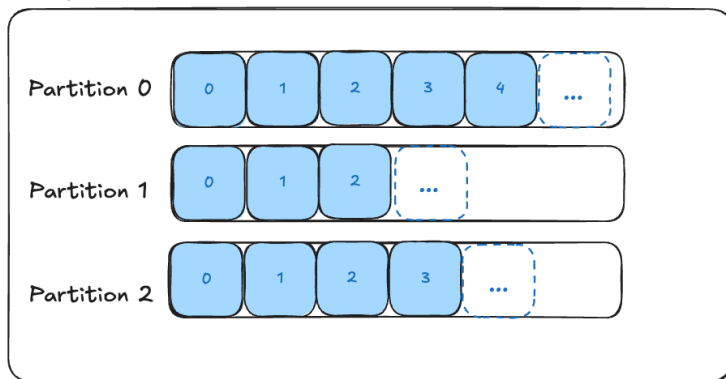
- **Netflix** — рекомендации в реальном времени во время просмотра;
- **Uber** — данные о пользователях, такси и поездках → прогноз спроса и surge pricing;
- **LinkedIn** — антиспам и рекомендации контактов по действиям пользователей.

Во всех примерах Kafka — **только транспорт**. Рекомендации, цены и антиспам считают приложения, которые читают из Kafka.

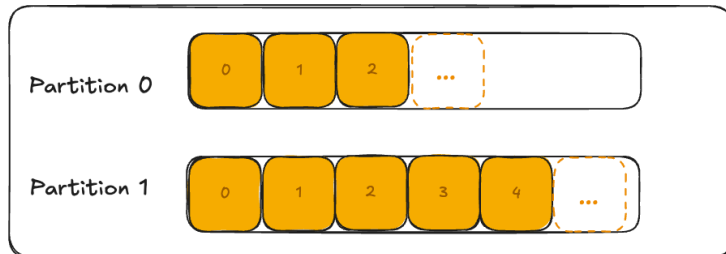
Модель данных — топики, партии, offsets

Иерархия: **Cluster** → **Topic** → **Partition** → **Message (offset)**. Кластер состоит из брокеров — серверов Kafka, хранящих партии (подробнее в [Брокеры и кластер](#)).

Topic «orders»



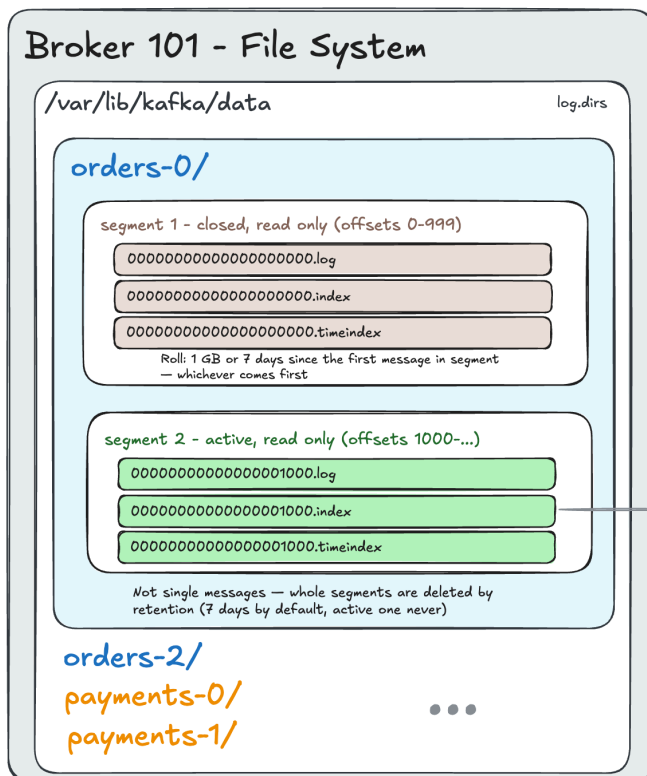
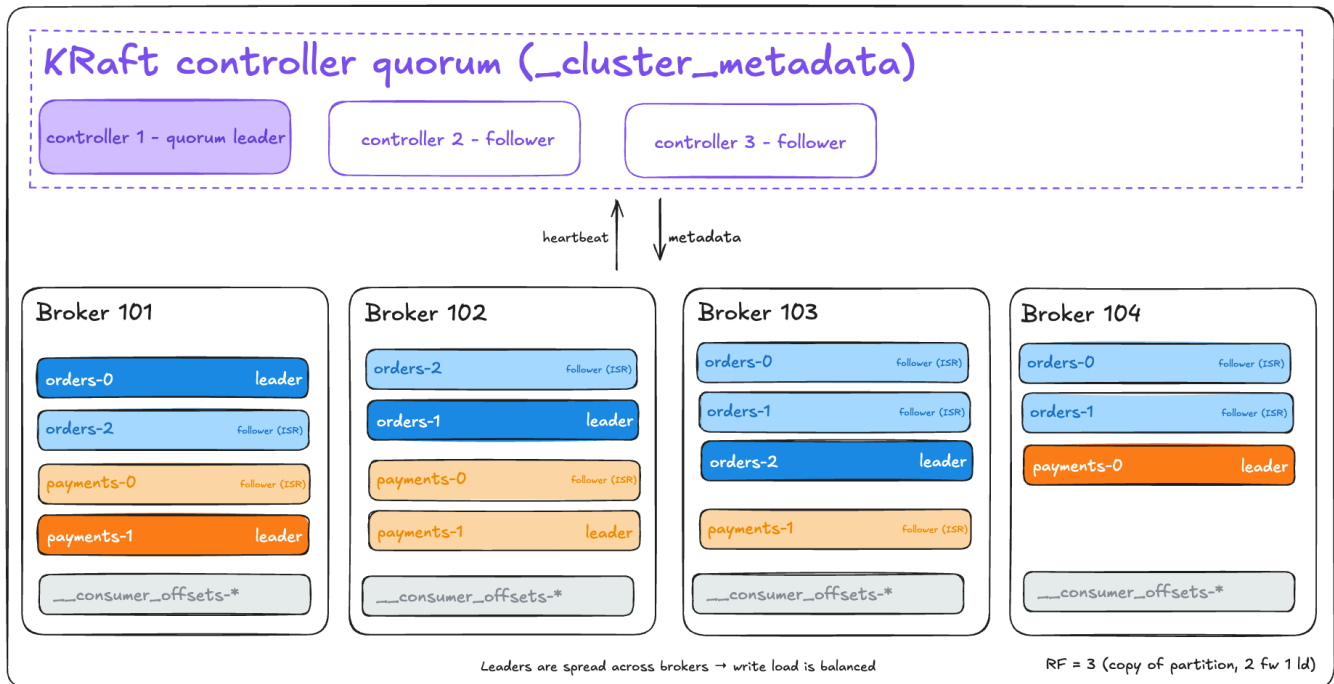
Topic «payments»



Message - orders

Key - binary (can be null) "user-42"	Value - binary (can be null) { "orderId": 981, "sum": 1490 }						
Headers (optional) <table border="1"> <thead> <tr> <th>key</th><th>value</th></tr> </thead> <tbody> <tr> <td>trace-id</td><td>ab12</td></tr> <tr> <td>source</td><td>web</td></tr> </tbody> </table>		key	value	trace-id	ab12	source	web
key	value						
trace-id	ab12						
source	web						
Timestamp (system or user set) 1791550094176							
offset (broker put it, in partition) 3							

Kafka Cluster



000000000000000000001000.log

Offset	Position (byte)	Key	Value	Headers	Timestamp
1000	0	"user-42"	{"orderId": 981, "sum": 1490}	trace-id: ab12 source: web	17915500949176
1001	131	"user-17"	{"orderId": 982, "sum": 320}	trace-id: c7f3 source: mobile	17915500949180
...	...	...	...	...	...
1042	5568	"user-42"	{"orderId": 1012, "sum": 870}	trace-id: e91d source: web	1791550096201

Indexes for fast lookup

000000000000000000001000.index

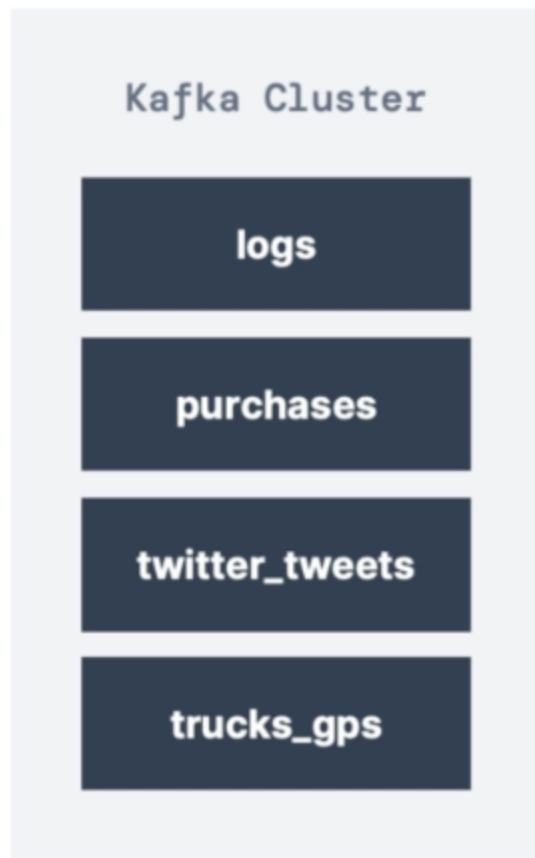
000000000000000000001000.timeindex

Relative offset	Position (byte)
0 (= 1000)	0
31 (= 1031)	4120
63 (= 1063)	8245

Timestamp	Relative offset
17915500949176	0 (= 1000)
1791550095697	31 (= 1031)
1791550097210	63 (= 1063)

Topic

Определение: топик (topic) — именованная логическая категория сообщений одного вида, основной способ организовать данные в Kafka. Последовательность сообщений топика во времени называют **data stream**. Ближайшая аналогия — таблица БД, но без схемы, связей и проверок.



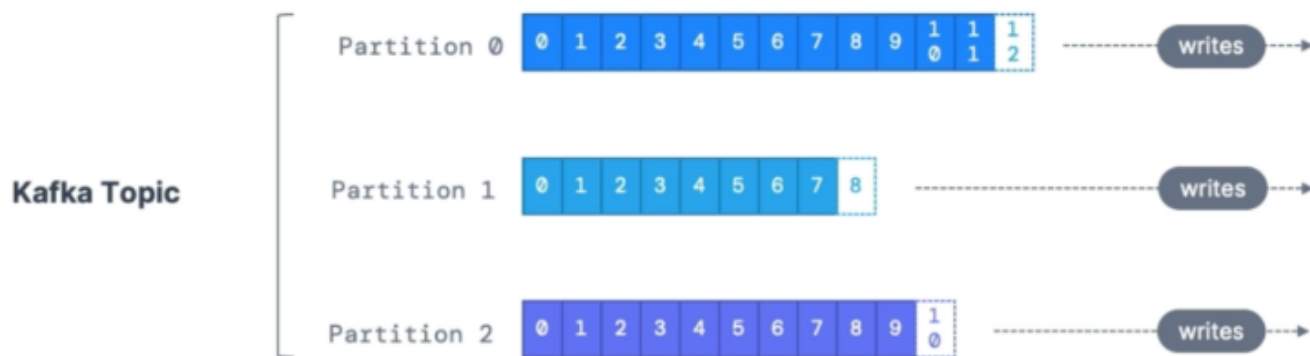
Логическая или реальная сущность? Топик реален как объект **метаданных**, но не как место хранения данных:

- топиков в кластере сколько угодно, идентифицируются по **имени** (`logs` , `purchases` , `trucks_gps`);
- формат любой (JSON, Avro, Protobuf, строки) — для Kafka это **массив байт**;
- запросами не опрашиваются (нет `SELECT`): пишут **producers**, читают **consumers**.

Partition и offset

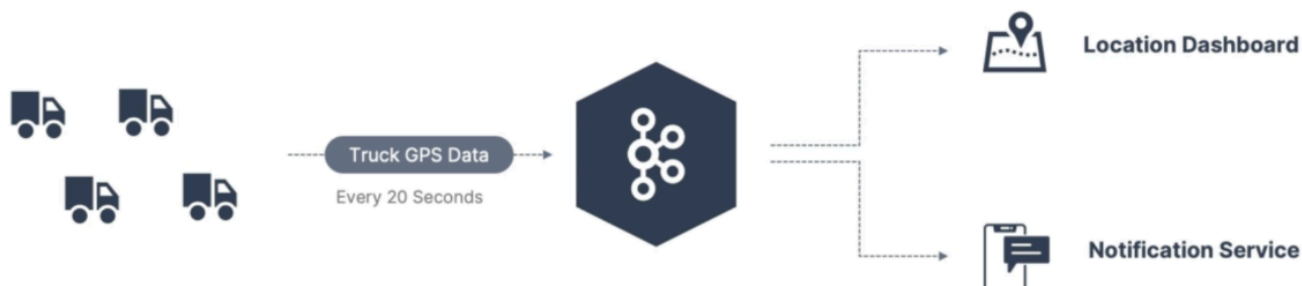
Определение: партиция (partition) — упорядоченная неизменяемая последовательность сообщений, на которые разделён топик. Физически — append-only лог на диске брокера: новые сообщения только дописываются в конец. Число партиций задаётся при создании топика.

Offset — порядковый номер сообщения внутри партиции: с 0, +1 на каждое новое.



- **порядок** гарантирован только **внутри партии**: offset уникален лишь в ней;
- **данные неизменяемы** и удаляются по сроку (`retention.ms` , по умолчанию 7 дней), а не после прочтения;
- **выбор партии**: без ключа — одна из партий, с ключом — $\text{hash}(\text{key}) \% \text{число_партий}$ → один ключ = одна партия = сохранённый порядок;
- **параллелизм**: партии распределены по брокерам и читаются consumers одновременно; слишком много партий нагружает кластер.

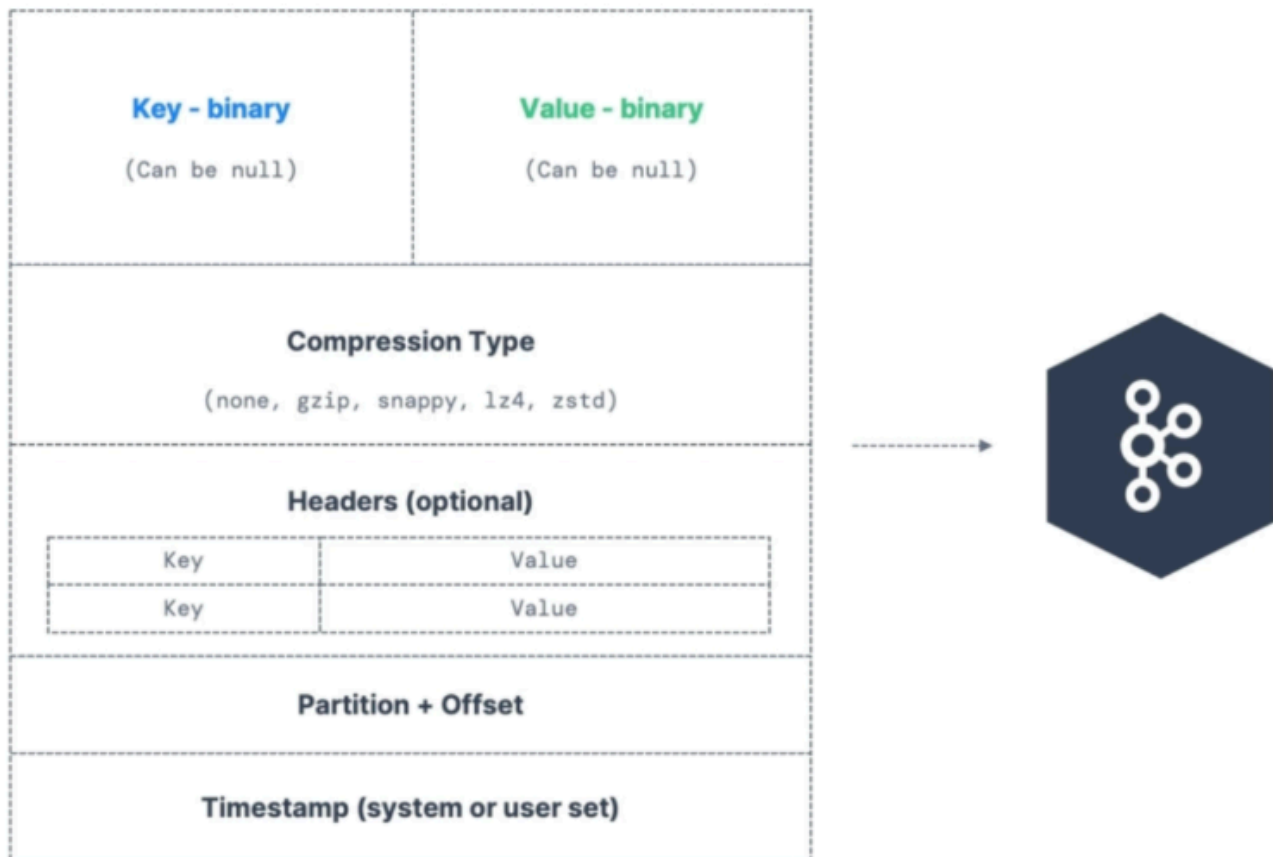
Пример: топик `trucks_gps`



- каждый грузовик раз в 20 с шлёт `{truck_id, широта, долгота}`;
- все позиции → один топик `trucks_gps` с 10 партиями (число произвольное);
- читают разные потребители: **Location Dashboard** (карта) и **Notification Service** (уведомления).

Анатомия сообщения

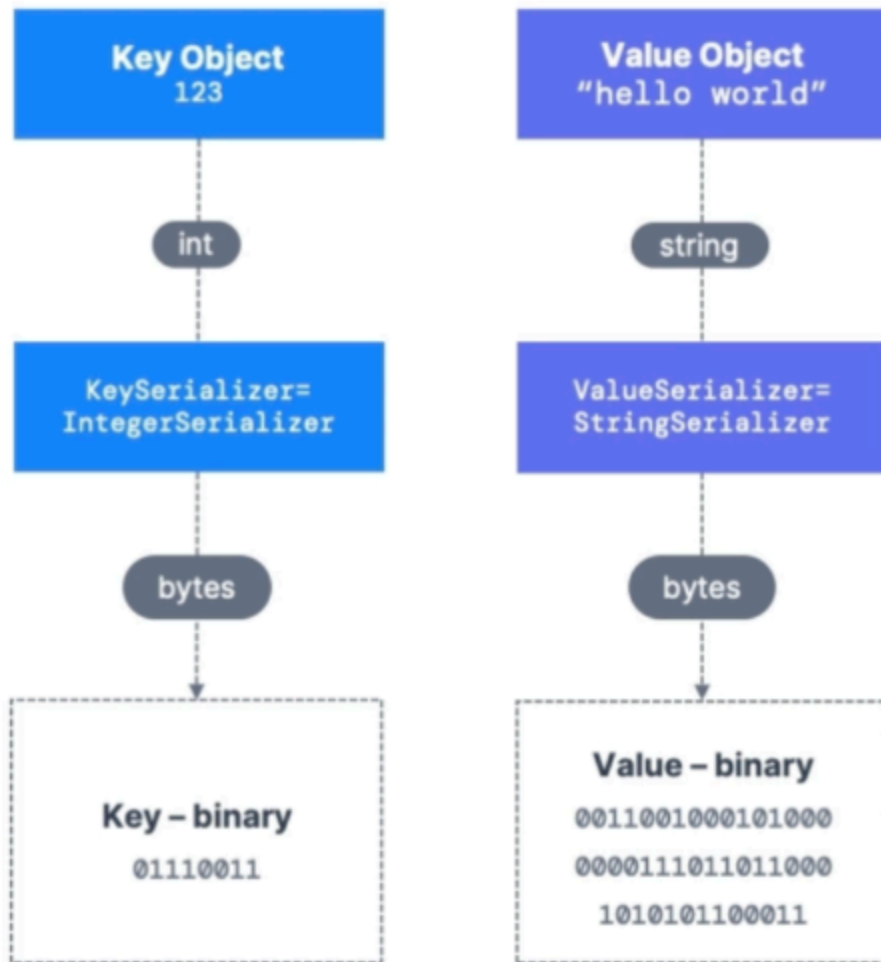
Определение: сообщение (message, record) — минимальная единица данных в Kafka. Producer формирует его из полей ниже, брокер дописывает в партию и назначает offset.



Поле	Описание
Key	бинарный, может быть <code>null</code>
Value	полезные данные, бинарные, может быть <code>null</code>
Compression type	<code>none</code> , <code>gzip</code> , <code>snappy</code> , <code>lz4</code> , <code>zstd</code>
Headers	необязательные пары «ключ — значение» с метаданными
Partition + Offset	куда записано; offset назначает брокер
Timestamp	ставится системой или пользователем

Сериализация и десериализация

Определение: **сериализация** — преобразование объекта в байты (на producer), **десериализация** — обратно (на consumer). Kafka принимает и отдаёт **только байты**.



- ключ и значение обрабатываются **отдельно**, своим (де)сериализатором:
 - key 123 (int) → IntegerSerializer → байты → IntegerDeserializer → 123 ;
 - value "hello world" → StringSerializer → байты → StringDeserializer → "hello world" ;
- распространённые: String (в т.ч. JSON), Int, Float, Avro, Protobuf;
- десериализатор consumer **обязан соответствовать** сериализатору producer — иначе мусор или `SerializationException` ;
- **формат нельзя менять за время жизни топика** — для нового формата создают **новый топик**.



Кластер — брокеры, репликация, координация

Брокеры и кластер

Определения:

- **Broker** — сервер Kafka: процесс, который хранит партиции на диске и обслуживает producers и consumers.
- **Kafka cluster** — группа совместно работающих брокеров.
- **Bootstrap server** — брокер, к которому клиент подключается первым. Им может быть **любой** брокер.
- **Metadata** — список брокеров, топиков, партиций и их размещения.



- брокер идентифицируется **целым ID** (`broker.id` , в KRaft — `node.id`); в курсе нумерация с 101;
- каждый брокер хранит **только часть** партиций; для старта хватает 3 брокеров, большие кластеры — 100+.

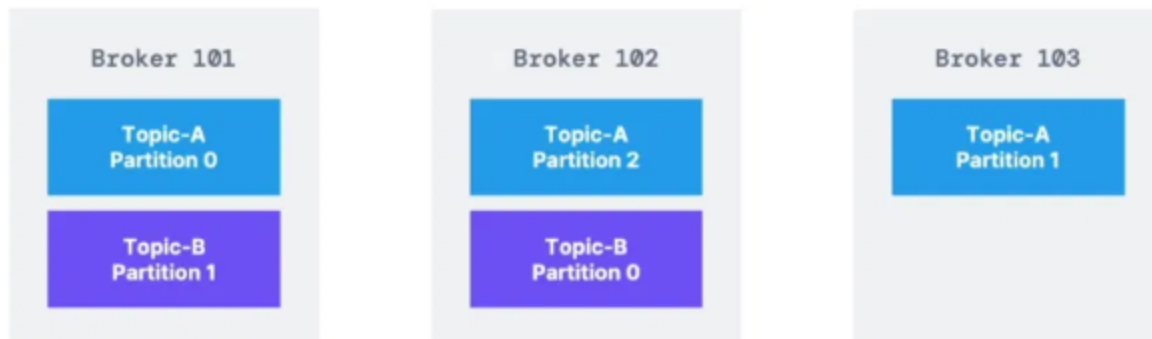
Распределение партиций

Topic-A (3 партиции) и Topic-B (2 партиции):

Брокер	Партиции
101	Topic-A P0, Topic-B P1

Брокер	Партиции
102	Topic-A P2, Topic-B P0
103	Topic-A P1

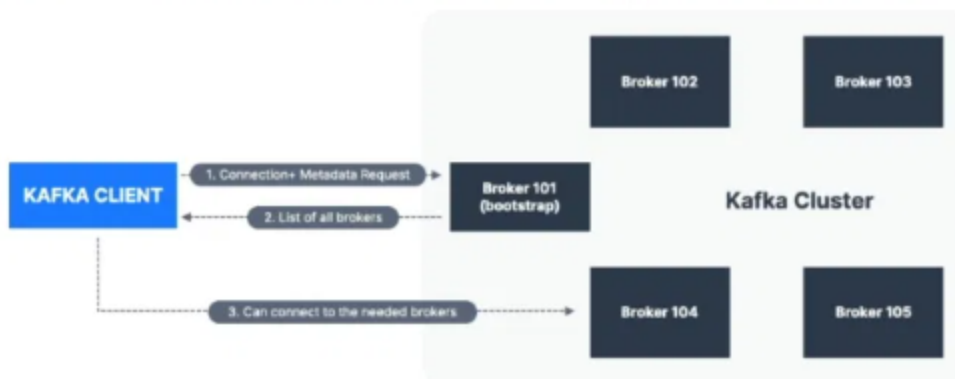
На 103 нет данных Topic-B — это нормально: данные и нагрузка делятся между серверами. Здесь у каждой партиции одна копия; копии — это репликация (ниже).



Broker discovery

Клиенты — **smart clients**: достаточно подключиться к одному брокеру, т.к. **каждый брокер знает метаданные всего кластера**.

1. Клиент подключается к bootstrap-брокеру → **Metadata Request**.
2. Брокер возвращает список всех брокеров и метаданные.
3. Клиент сам подключается к брокерам с нужными партициями.



```
// несколько адресов — на случай недоступности первого; весь кластер
перечислять не нужно
props.put("bootstrap.servers", "broker101:9092,broker102:9092");
```

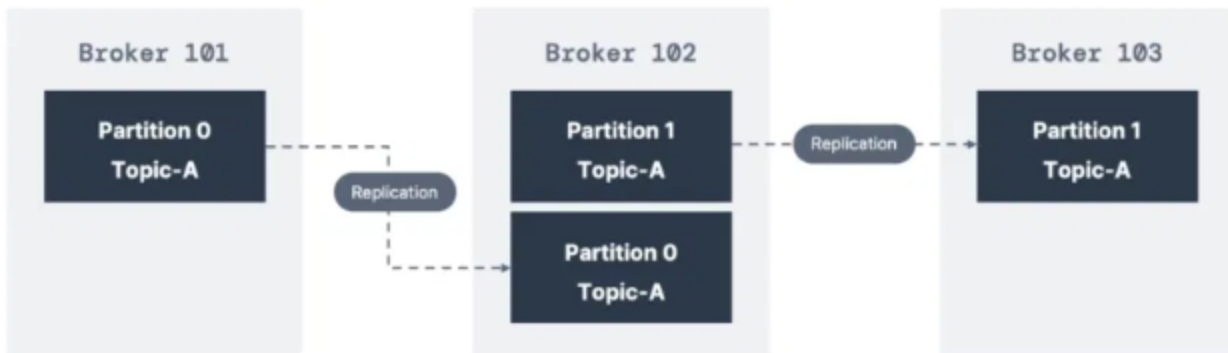
Репликация — replication factor, leader, ISR

Определения:

- **Replication factor (RF)** — сколько копий каждой партии хранится в кластере; копии лежат на **разных** брокерах.
- **Реплика** — одна копия партии на конкретном брокере.
- **Leader** — реплика, через которую идёт запись; у партии в любой момент **ровно один** лидер.
- **ISR (in-sync replica)** — реплика, которая успевает копировать данные с лидера.

Replication factor

- RF должен быть > 1 , обычно 2–3; в продакшене стандарт — **RF = 3**;
- $RF \leq$ числа брокеров;
- пример: Topic-A, 2 партии, RF = 2 $\rightarrow$ P0 на 101 + копия на 102, P1 на 102 + копия на 103.



Упал Broker 102 $\rightarrow$ данные всё ещё доступны: P0 есть на 101, P1 — на 103.



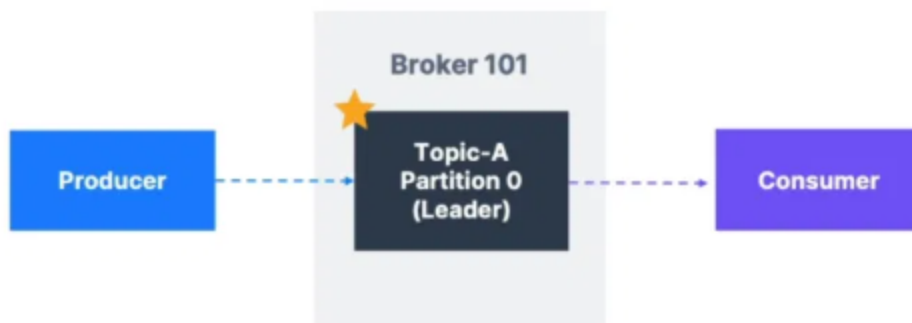
Leader и ISR

- producer пишет **только в лидера**, остальные реплики копируют с него $\rightarrow$ у партии **один leader и несколько ISR**;
- лидер упал $\rightarrow$ controller выбирает нового лидера **из ISR**; клиенты узнают о нём из метаданных (отсюда «producer/consumer умеют восстанавливаться»);

- реплика выпадает из ISR, если отстаёт дольше `replica.lag.time.max.ms` (по умолчанию 30 с).



- **producer** — пишет только в лидера;
- **consumer** — по умолчанию читает из лидера.



Replica fetching (Kafka 2.4+)

Consumer можно настроить читать из **ближайшей реплики** — ниже задержка и дешевле межзонный трафик в облаке. Запись — по-прежнему только в лидера.

```

# брокер
broker.rack=eu-1a
replica.selector.class=org.apache.kafka.common.replica.RackAwareReplicaSelector

# консьюмер
client.rack=eu-1a
  
```



acks и durability — гарантии записи

Определения:

- **Acknowledgement (ack)** — подтверждение брокера, что запись принята.
- **acks** — настройка producer: сколько реплик должно подтвердить запись.
- **Durability** — способность топика сохранить данные при потере брокеров.

acks

acks	Ждёт подтверждения	Риск	Почему
0	ни от кого	возможна потеря	producer не узнает, что брокер упал до записи
1	только лидер	ограниченная потеря	лидер подтвердил и упал до репликации → у нового лидера данных нет
all (-1)	лидер + все ISR	без потери	данные уже на всех ISR

С **Kafka 3.0** по умолчанию `acks=all` и `enable.idempotence=true` (до 3.0 — `acks=1`).



```
props.put("acks", "all"); // "0", "1", "all"
```

acks=all + min.insync.replicas

- `acks=all` ждёт **текущие ISR**, а не все реплики: если ISR сжался до лидера, это фактически `acks=1`;
- `min.insync.replicas` (топик/брокер) — минимум ISR, включая лидера, при котором запись разрешена; меньше → `NotEnoughReplicasException`;
- надёжная связка: **RF = 3**, `acks=all`, `min.insync.replicas=2` — переживает падение одного брокера без потери данных и без остановки записи.

Durability топика

- **RF = N** → можно потерять до **N - 1** брокеров и сохранить данные (RF = 3 → 2 брокера);
- это про сохранность; чтобы ещё и **продолжать писать** с `acks=all`, живых ISR должно быть $\geq \text{min.insync.replicas}$.



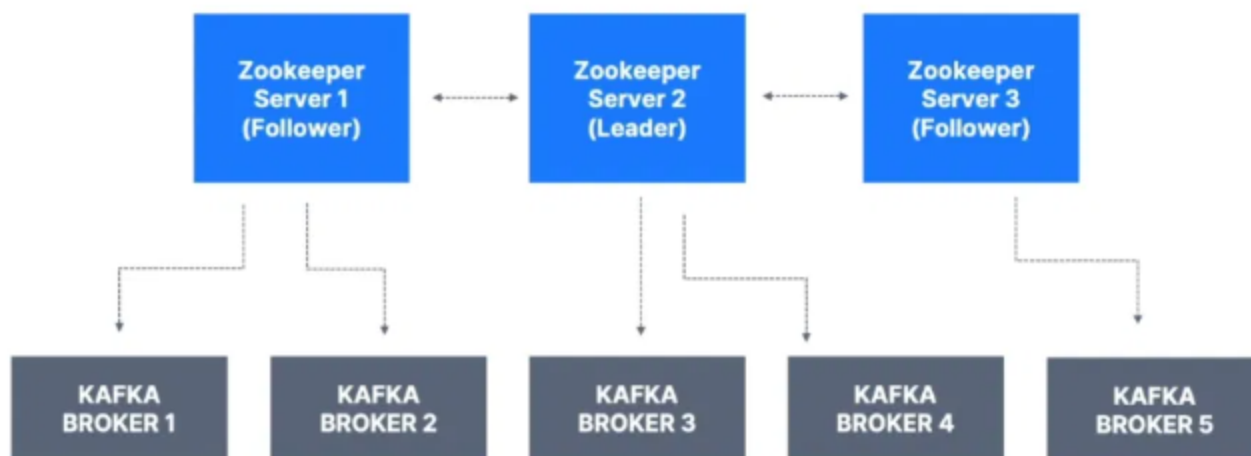
Zookeeper и KRaft — кто управляет кластером

Определения:

- **Zookeeper** — отдельная распределённая система хранения метаданных и координации; в старых версиях управляла брокерами Kafka.
- **Zookeeper ensemble** — кластер серверов Zookeeper.
- **KRaft (Kafka Raft)** — режим Kafka без Zookeeper (KIP-500): метаданные хранят и согласуют сами узлы-контроллеры по протоколу **Raft**.
- **Quorum controller** — группа контроллеров Kafka, управляющая метаданными; **quorum leader** принимает изменения, остальные реплицируют.

Что делал Zookeeper

- хранит список брокеров, помогает с **leader election** партиций;
- уведомляет Kafka об изменениях: новый/удалённый топик, брокер упал/поднялся;
- ансамбль: один **leader** (записи) + **followers** (чтение), **нечётное** число серверов (1, 3, 5, 7) — из-за **кворума**: 3 сервера переживают потерю 1, 5 — 2, четвёртый к трём устойчивости не добавляет;
- с Kafka 0.10 consumer offsets в Zookeeper **не хранятся** (они в `__consumer_offsets`).



Версии

Версия	Zookeeper
Kafka 2.x	обязателен
Kafka 3.x	опционален, можно KRaft; KRaft production-ready с 3.3
Kafka 4.x (март 2025)	удалён , только KRaft



APACHE ZooKeeper™

В курсе: «Zookeeper нужен с брокерами, KRaft пока не для продакшена» — устарело.
Новые кластеры поднимают на KRaft.

Zookeeper и клиенты

- клиенты и CLI давно ходят **в брокеры**, не в Zookeeper: offsets в Kafka с 0.10, `kafka-topics.sh --bootstrap-server` с 2.2, `--zookeeper` удалён в 3.0 → для клиентов переход на KRaft незаметен;
- Zookeeper менее защищён: его порты открыты **только брокерам**;
- **никогда не указывай Zookeeper в конфиге клиентов.**

```
kafka-topics.sh --bootstrap-server localhost:9092 --list # ✓  
kafka-topics.sh --zookeeper localhost:2181 --list # ✗ удалено в  
Kafka 3.0
```

Зачем KRaft

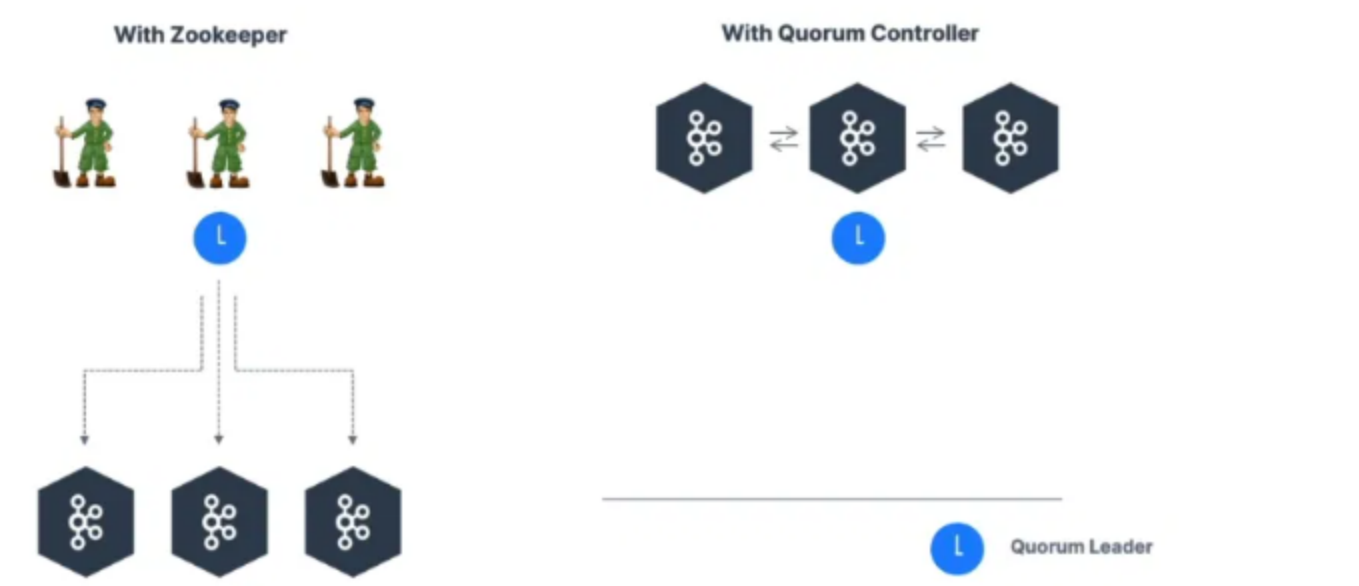
Zookeeper плохо масштабируется при **> 100 000 партиций** (работа над KIP-500 — с 2020).
Без него Kafka:

- масштабируется до **миллионов партиций**;
- стабильнее, проще мониторить и администрировать;
- имеет **единую модель безопасности**;
- запускается **одним процессом**;
- быстрее выключает контроллер и восстанавливается.

Архитектура KRaft

- с **Zookeeper**: две системы — отдельный кластер Zookeeper со своим лидером управляет брокерами;
- с **KRaft**: контроллеры — **сами узлы Kafka**, один из них quorum leader;
- метаданные — во внутреннем топике `__cluster_metadata` (реплицируемый лог); каждый контроллер держит их у себя → новому лидеру не нужно загружать их заново.

```
process.roles=broker,controller # узел совмещает обе роли (разработка)
# process.roles=controller      # только контроллер
# process.roles=broker          # только брокер (в продакшене роли
                                # разделяют)
```

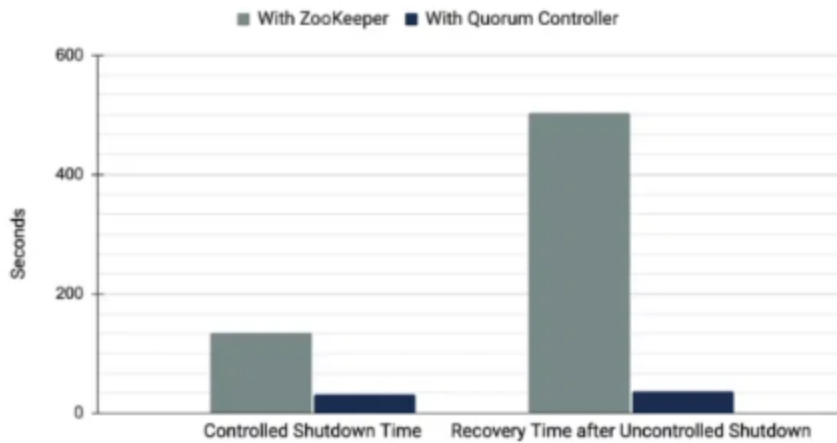


Производительность (2 млн партиций)

Операция	Zookeeper	KRaft
Controlled shutdown	≈ 135 с	≈ 30 с
Recovery после аварии	≈ 500 с	≈ 35 с

Значения примерные, с графика. Источник: [Confluent: Kafka without ZooKeeper — a sneak peek](#)

Timed Shutdown Operations In Apache Kafka with 2 Million Partitions
Faster is better



Окружение — установка, CLI, Java-проект

Установка Kafka

Определения:

- **Kafka binaries** — официальный архив Apache: серверы + CLI-скрипты.
- **Kafka CLI tools** — консольные утилиты из того же архива.
- **Conduktor** — внешний инструмент с UI для запуска Kafka и просмотра топиков.
- **WSL2** — подсистема Linux в Windows; Kafka в ней работает как на Linux.

Схема везде одна: **запустить Kafka + поставить CLI tools**. Для разработки достаточно **одного брокера на localhost** (тогда RF может быть только 1).

ОС	Путь
Mac	Conduktor или бинарники / <code>brew</code>
Linux	Conduktor или бинарники
Windows 10 (2004+) / 11	WSL2 + бинарники или Conduktor; без WSL не рекомендуется

В курсе рекомендуется запуск с Zookeeper, KRaft «только для разработки». В Kafka 4.x — только KRaft.

Быстрый старт сейчас

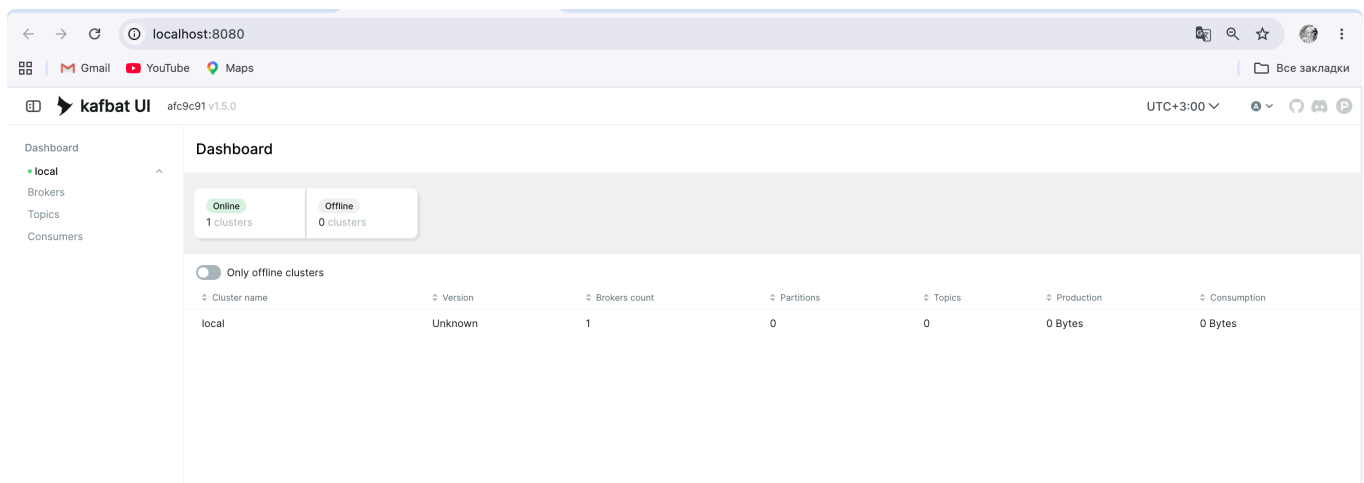
```
# Docker — проще всего на любой ОС, официальный образ сразу в KRaft
docker run -d --name kafka -p 9092:9092 apache/kafka:latest
```

```
# бинарники Kafka 4.x (Linux / Mac / WSL2)
KAFKA_CLUSTER_ID="$(bin/kafka-storage.sh random-uuid)"
bin/kafka-storage.sh format --standalone -t $KAFKA_CLUSTER_ID -c
config/server.properties
bin/kafka-server-start.sh config/server.properties

# проверка
bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
```

Kafka UI

Kafka UI (проект **kafbat**) — бесплатный open-source веб-интерфейс: топики, сообщения, consumer groups, lag. Бесплатная альтернатива Conduktor. Открывается на <http://localhost:8080>, кластер добавляется в интерфейс.



Kafka CLI — как вызывать

Определение: **Kafka CLI** — консольные скрипты для топики, producers, consumers и групп из комплекта бинарников. Адрес брокера всегда передаётся через **--bootstrap-server** (не **--zookeeper**).

Имя команды	Установка
kafka-topics.sh	бинарники (Linux, Mac, WSL2)
kafka-topics.bat	бинарники на Windows без WSL2
kafka-topics	homebrew, apt

- `command not found` → не настроен `$PATH`; вызывай по полному пути: `kafka_2.13-3.0.0/bin/kafka-topics.sh` (2.13 — версия Scala, 3.0.0 — Kafka).

CLI в Docker

В `apache/kafka` скрипты лежат в `/opt/kafka/bin`.

```
docker exec -it kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server
localhost:9092 --list

# или зайти в контейнер
docker exec -it kafka bash
cd /opt/kafka/bin && ./kafka-topics.sh --bootstrap-server localhost:9092 --
list
```

Скрипт-обёртка в проекте — файл `kafka` рядом с `docker-compose.yml`, сам подставляет `--bootstrap-server`:

```
#!/bin/sh
cmd="$1"; shift
exec docker exec -it kafka /opt/kafka/bin/kafka-"$cmd".sh --bootstrap-server
localhost:9092 "$@"
```

```
chmod +x kafka # один раз
./kafka topics --list
./kafka console-producer --topic test
./kafka console-consumer --topic test --from-beginning
```

Альтернатива — функция в `~/.zshrc` (без автоподстановки адреса):

```
kafka() { docker exec -it kafka /opt/kafka/bin/kafka-"$1".sh "${@:2}"; }
kafka topics --bootstrap-server localhost:9092 --list
```

Дальше команды даны в полном виде; через `./kafka` то же самое без `kafka-...sh` и `--bootstrap-server`.

kafka-topics.sh — управление топиками

```
kafka-topics.sh --bootstrap-server localhost:9092 --list
# СПИСОК
```

```
kafka-topics.sh --bootstrap-server localhost:9092 --topic first_topic --
create                                # партиции/RF из настроек брокера (по умолчанию 1)
kafka-topics.sh --bootstrap-server localhost:9092 --topic first_topic --
create --partitions 3 --replication-factor 1
# --replication-factor 2 при одном брокере →
InvalidReplicationFactorException

kafka-topics.sh --bootstrap-server localhost:9092 --topic first_topic --
describe                             # партиции, лидер, реплики, ISR
kafka-topics.sh --bootstrap-server localhost:9092 --topic first_topic --
alter --partitions 5
kafka-topics.sh --bootstrap-server localhost:9092 --topic first_topic --
delete
```

- RF не может превышать число брокеров;
- партиции можно только **увеличить**; после этого `hash(key) % numPartitions` меняется → ключи переезжают, порядок по ключу ломается.

Kafka SDK и Java-проект

SDK

- **официальный** клиент — только **Java** (`kafka-clients`); остальные (Go, Python, .NET, Rust, JS...) поддерживает сообщество — [список](#);
- почему Java: Kafka написана на JVM-языках (брокер исторически на Scala, клиенты на Java), Java-клиент живёт в одном репозитории с брокером → фичи появляются в нём первыми; Kafka Streams и Connect — только JVM;
- многие клиенты других языков — обёртки над C-библиотекой **librdkafka**;
- в Java-проектах обычно **Spring for Apache Kafka** (`KafkaTemplate` , `@KafkaListener`) — надстройка над теми же `KafkaProducer` / `KafkaConsumer` .

Настройка проекта

IntelliJ IDEA (Community), **Java 21**, **Gradle Kotlin DSL**. Зависимости: `kafka-clients` + реализация SLF4J (без неё клиент не пишет логи).

```
// build.gradle.kts
plugins {
    java
}
```

```

group = "io.conduktor.demos"
version = "1.0-SNAPSHOT"

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21) // сборка на JDK 21
        независимо от системной JDK
    }
}

repositories {
    mavenCentral()
}

dependencies {
    implementation("org.apache.kafka:kafka-clients:4.3.1") // та же
    версия, что у брокера
    implementation("org.slf4j:slf4j-simple:2.0.17") // реализация
    логгера; slf4j-api подтянется сам
}

```

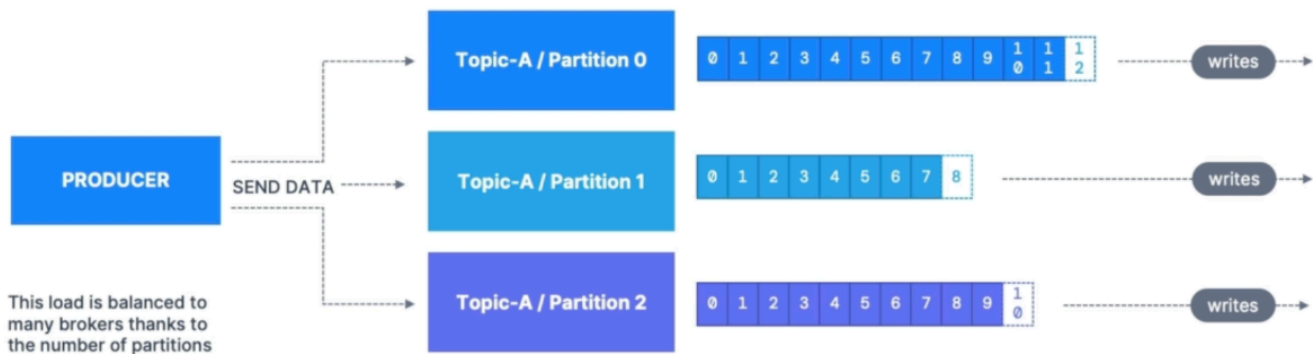
- `slf4j-api` — интерфейс, `slf4j-simple` — реализация, пишет в консоль;
- чтобы Gradle сам скачивал JDK для `toolchain`, в `settings.gradle.kts` нужен плагин `org.gradle.toolchains.foojay-resolver-convention` (IntelliJ обычно добавляет его сам);
- минимум для клиентов Kafka 4.x — Java 11.

Producer

Как producer пишет данные — партии и ключи

Определения:

- **Producer** — клиентское приложение, которое пишет сообщения в топики.
- **Ключ (message key)** — необязательное поле, по которому выбирается партия.
- **Partitioner** — логика внутри producer, выбирающая партию для сообщения.
- **Key hashing** — способ, которым partitioner сопоставляет ключ с партией.



Producer пишет в **партиции** топика. Он заранее знает, в какую партицию и на какой брокер писать, и шлёт сообщение сразу туда → нагрузка распределяется по брокерам. При отказе брокера producer восстанавливается сам.

Ключи

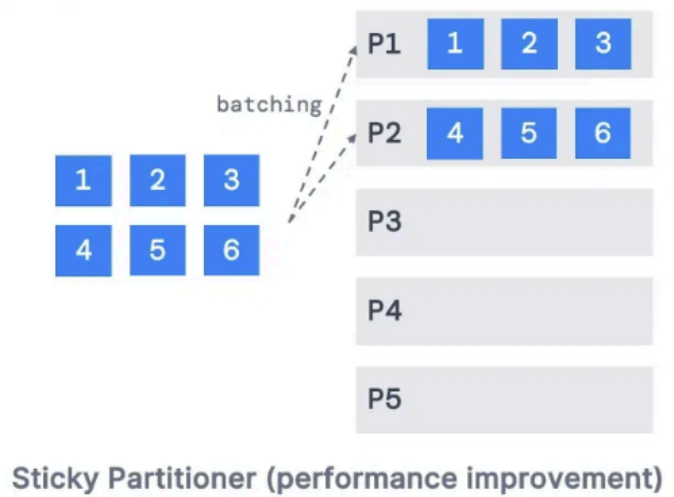
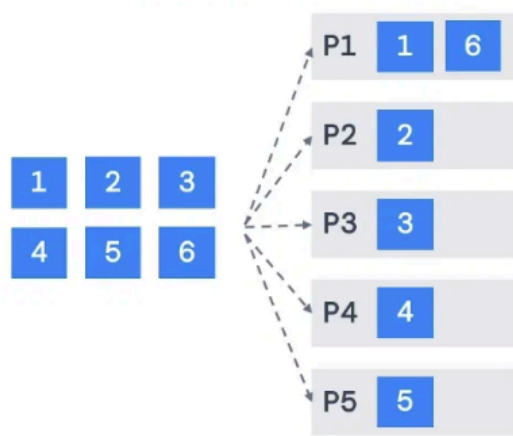


Ключ	Куда попадает сообщение
key = null	распределяется по партициям (в курсе — round robin, сейчас — sticky)
key != null	всегда в одну и ту же партицию (хеш ключа)

Ключ задают, когда важен **порядок по полю**: с ключом `truck_id` все позиции грузовика лягут в одну партицию и прочтутся по порядку.

Round robin vs sticky partitioner

- **Round robin**: сообщения по одному в P0, P1, P2, P0...
- **Sticky (с Kafka 2.4, по умолчанию для key = null)**: producer набивает **батч** в одну партицию и только потом переключается. Меньше запросов к брокеру, на большом объёме распределение всё равно равномерное.
- С Kafka 3.3 (KIP-794) партиция меняется, когда в неё набралось `~ batch.size` байт и батч отправлен.



Хеширование ключа



`send()` → `partitioner` → для записи с ключом считается хеш **murmur2**:

```
// на слайде (упрощено и неточно)
targetPartition = Math.abs(Utils.murmur2(keyBytes)) % (numPartitions - 1)

// в исходниках Kafka
targetPartition = Utils.toPositive(Utils.murmur2(keyBytes)) % numPartitions
```

Следствие: **изменил число партиций** → тот же ключ может попасть в другую партицию, гарантия порядка по ключу нарушается.

kafka-console-producer.sh

Каждая строка ввода — одно сообщение, выход — `Ctrl + C`.

```
# без ключей
kafka-console-producer.sh --bootstrap-server localhost:9092 --topic
first_topic
> Hello World
> I love Kafka

# с настройками producer
```

```
kafka-console-producer.sh --bootstrap-server localhost:9092 --topic
first_topic --producer-property acks=all

# с ключами: до ":" — ключ, после — значение
kafka-console-producer.sh --bootstrap-server localhost:9092 --topic
first_topic \
    --property parse.key=true --property key.separator=:
> example key:example value
> name:Stephane

# по одному сообщению в каждую партицию (чтобы увидеть распределение без
ключей)
kafka-console-producer.sh --bootstrap-server localhost:9092 --topic
first_topic \
    --producer-property
partitioner.class=org.apache.kafka.clients.producer.RoundRobinPartitioner
```

- в режиме с ключами строка без разделителя → No key separator found;
- **несуществующий топик создаётся автоматически** с num.partitions брокера (по умолчанию 1); управляется auto.create.topics.enable (по умолчанию true, в продакшене выключают) → **создавай топики заранее**;
- в Docker вместо правки server.properties — KAFKA_NUM_PARTITIONS: 3 в environment сервиса в docker-compose.yml;
- без ключей console-producer часто шлёт всё в одну партицию (sticky, батч ~16 KB) — для демо распределения нужен RoundRobinPartitioner;
- KIP-1147 переименовывает флаги: --property → --reader-property, --producer-property → --command-property (старые работают с предупреждением).

Java Producer — базовый

Определения:

- `KafkaProducer<K, V>` — класс producer: подключается к брокерам и отправляет записи.
- `ProducerRecord<K, V>` — запись: топик, значение, опционально ключ, партиция, headers.
- `ProducerConfig` — константы имён настроек (`BOOTSTRAP_SERVERS_CONFIG` и др.).

Жизненный цикл:

Шаг	Что делает
config	bootstrap.servers + сериализаторы
new KafkaProducer	создать producer
send(record)	асинхронно : кладёт запись в буфер и сразу возвращается
flush()	синхронно ждёт отправки всего буфера
close()	flush + закрытие

```

public class ProducerDemo {

    private static final Logger log =
LoggerFactory.getLogger(ProducerDemo.class);

    public static void main(String[] args) {
        Map<String, Object> config = Map.of(
            ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092"
        );

        // сериализаторы объектами → проверка типов при компиляции,
        KafkaProducer<String, String> выводится сам
        try (var producer = new KafkaProducer<>(config, new
StringSerializer(), new StringSerializer())) {
            var record = new ProducerRecord<String, String>("demo_java",
"hello world");
            producer.send(record);    // асинхронно
        }                            // try-with-resources: close() = flush
+ закрытие, даже при исключении
    }
}

```

```

./kafka console-consumer --topic demo_java --from-beginning
> hello world

```

Topics / demo_java

Produce Message

Overview

Messages

Consumers

Settings

Statistics

ACLs

Newest

Select partitions

Key Serde

Value Serde

Refresh

Search

+ Add Filters

13 ms

11 Bytes

1 messages consumed

	Offset	Partition	Timestamp	Key Preview	Value Preview
+	0	0	10/9/2026, 15:35:20.393		hello world

- без `flush()` / `close()` программа может завершиться **раньше**, чем сообщение уйдёт из буфера;
- топик создай заранее: `./kafka topics --create --topic demo_java --partitions 3`.

Callback и sticky partitioner

Определения:

- **Callback** — функциональный интерфейс `onCompletion(RecordMetadata metadata, Exception e)`; второй аргумент `send()`. Вызывается, когда запись подтверждена или упала с ошибкой.
- **RecordMetadata** — `topic()`, `partition()`, `offset()`, `timestamp()` записанного сообщения.

```
Map<String, Object> config = Map.of(
    ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092",
    ProducerConfig.BATCH_SIZE_CONFIG, 400 // только для демо:
    маленький батч → чаще смена партиии
    // , ProducerConfig.PARTITIONER_CLASS_CONFIG,
    RoundRobinPartitioner.class.getName() // «как round robin»
);

try (var producer = new KafkaProducer<>(config, new StringSerializer(), new
StringSerializer())) {
    for (int j = 0; j < 10; j++) { // 10 пачек
        for (int i = 0; i < 30; i++) { // по 30 сообщений
            var record = new ProducerRecord<String, String>("demo_java",
"hello world " + i);
```

```

        producer.send(record, (metadata, e) -> {
            if (e == null) {
                log.info("topic={} partition={} offset={} timestamp={}",
                    metadata.topic(), metadata.partition(),
                    metadata.offset(), metadata.timestamp());
            } else {
                log.error("Error while producing", e);
            }
        });
    }

    try {
        Thread.sleep(500); // пауза: батч успевает уйти
на брокер
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt(); // восстановить флаг, не
глотать
        return;
    }
}
}

```

- callback выполняется в I/O-потоке `kafka-producer-network-thread`, не в `main` → долгую работу в нём не делают, иначе тормозит вся отправка;
- `send()` возвращает `Future<RecordMetadata>`; `send(record).get()` — синхронная отправка, в цикле так не делают;
- плейсхолдеры SLF4J `{}` вместо конкатенации: строка собирается, только если уровень включён.

Результат

Без пауз (30 сообщений за миллисекунды) — все в одну партицию, offsets подряд:

```

[kafka-producer-network-thread | producer-1] INFO ProducerDemoWithCallback -
topic=demo_java partition=0 offset=1 timestamp=1791550002384
...
[kafka-producer-network-thread | producer-1] INFO ProducerDemoWithCallback -
topic=demo_java partition=0 offset=30 timestamp=1791550002388

```

С паузами (код выше) — партиция меняется только **между** пачками:

```

... partition=0 offset=31 timestamp=1791550094176    ← пачка 1 → P0
... partition=2 offset=0  timestamp=1791550094685    ← пачка 2 → P2 (через
~500 мс)

```

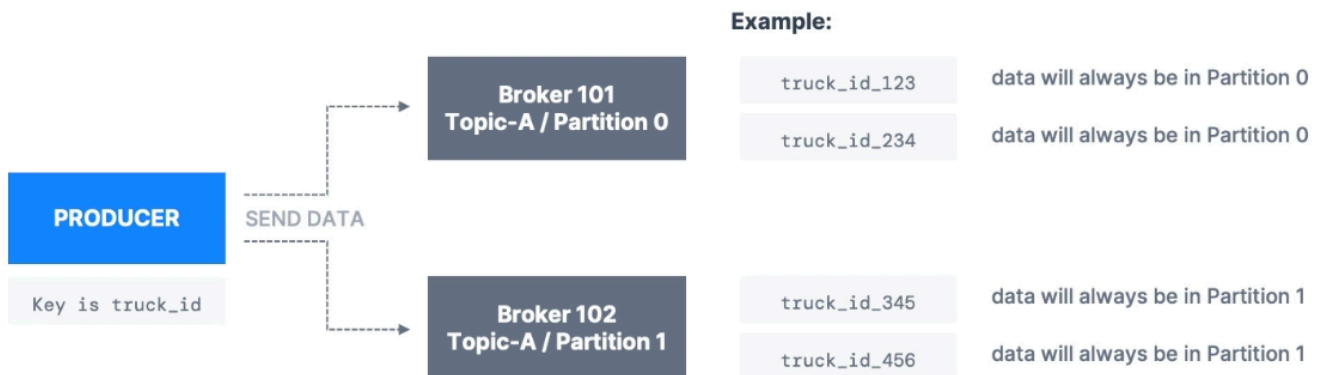
```
... partition=1 offset=0    timestamp=1791550096201    ← пачка 5 → P1
... partition=0 offset=61   timestamp=1791550096706    ← пачки 6–10 → снова P0
```

Пачки	Партиция	Offsets
1	0	31–60
2–4	2	0–89
5	1	0–29
6–10	0	61–210

- пачка целиком уходит в одну партицию; переключение не после каждой пачки → на коротком отрезке распределение неравномерное;
- скачок `timestamp` на ~500 мс при смене партиции — это пауза;
- в логе `ProducerConfig values` видны дефолты Kafka 4.x: `acks = -1`, `enable.idempotence = true`, `linger.ms = 5`.

Producer с ключами

Одинаковый ключ → одна и та же партиция. Пример со слайда: `truck_id_123`, `truck_id_234` → всегда P0; `truck_id_345`, `truck_id_456` → всегда P1.



```
try (var producer = new KafkaProducer<>(config, new StringSerializer(), new
StringSerializer())) {
    for (int round = 0; round < 2; round++) {                // 2 прохода по тем
же ключам
        for (int i = 0; i < 10; i++) {
            String key = "id_" + i;
            var record = new ProducerRecord<>("demo_java", key, "hello world
" + i);    // топик, ключ, значение
```

```

        producer.send(record, (metadata, e) -> { // key effectively
final внутри итерации → можно в лямбде
            if (e == null) {
                log.info("key={} partition={} offset={}", key,
metadata.partition(), metadata.offset());
            } else {
                log.error("Error while producing", e);
            }
        });
    }
    producer.flush();
}
}

```

Результат (3 партии):

Партия	Ключи
0	id_1, id_3, id_6
1	id_0, id_8
2	id_2, id_4, id_5, id_7, id_9

- во втором проходе каждый ключ — в **ту же** партию, меняется только offset;
- распределение неравномерное: хеш не гарантирует «поровну» на малом числе ключей;
- совпадает с курсом: murmur2 детерминирован, при том же числе партий результат одинаков на любой машине.

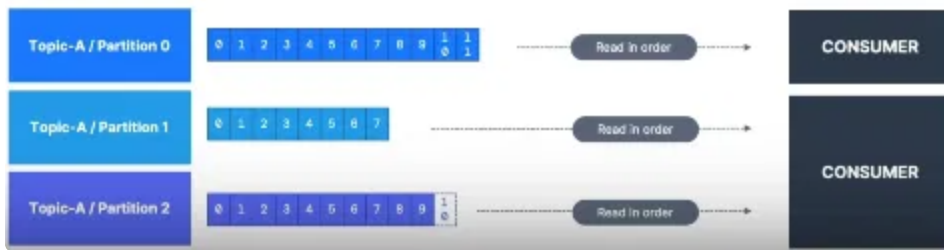
Consumer

Как consumer читает данные

Определение: Consumer — клиентское приложение, которое читает данные из топика (по имени).

- **pull-модель:** consumer сам запрашивает данные через `poll()`, брокер ничего не пушит → читает в своём темпе, медленный потребитель не перегружается;
- **знает, откуда читать:** по метаданным читает каждую партию у её лидера; при падении брокера перезапрашивает метаданные и продолжает у нового лидера;

- **порядок:** от меньшего offset к большему, **только внутри партиции**; между партициями порядок не гарантирован;
- один consumer может читать несколько партиций;
- нужен порядок по сущности (например, заказу) → общий ключ → одна партиция.



kafka-console-consumer.sh

Выход — Ctrl + C.

```
# только НОВЫЕ сообщения (после запуска)
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic
first_topic

# с начала топика
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic
first_topic --from-beginning

# с ключом, партицией и timestamp
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic
first_topic \
    --property print.timestamp=true --property print.key=true --property
print.partition=true \
    --from-beginning
```

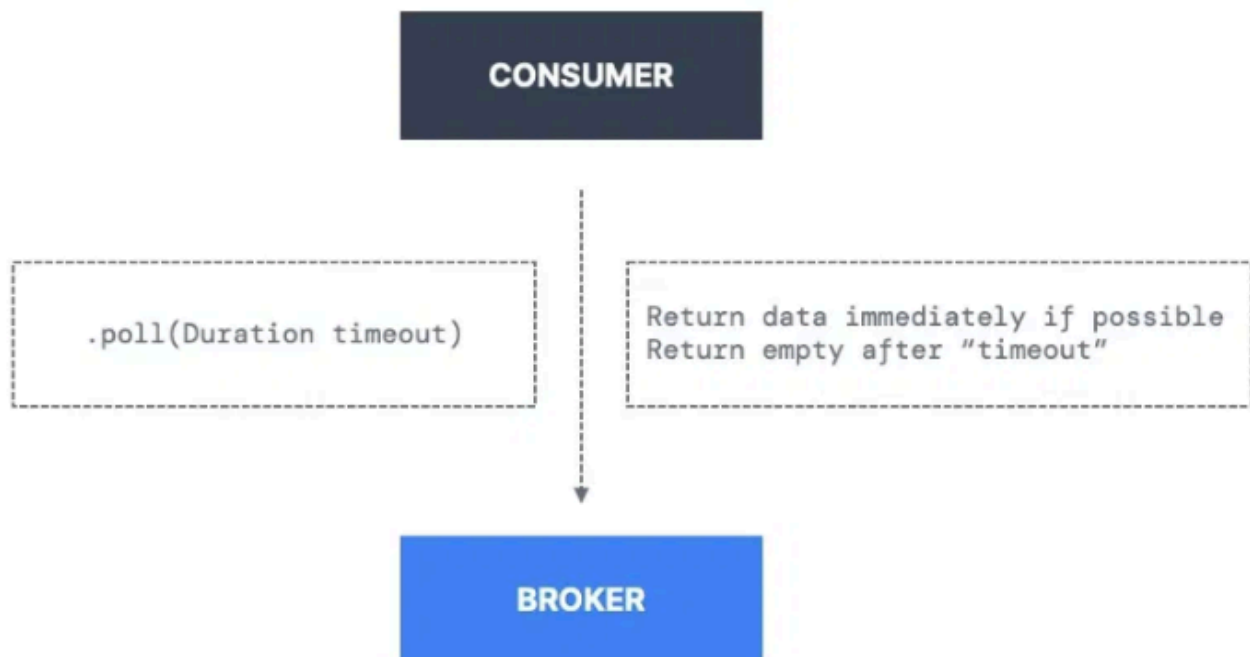
- при `--from-beginning` сообщения разных партиций идут вперемешку — порядок только внутри партиции;
- `print.partition=true` — видно, как ключи раскладываются по партициям;
- KIP-1147: `--property` у consumer → `--formatter-property` (старый работает с предупреждением).

В курсе: `--formatter kafka.tools.DefaultMessageFormatter`. В Kafka 4.x этот класс удалён (переехал в `org.apache.kafka.tools.consumer`), а `DefaultMessageFormatter` и так используется по умолчанию — `--formatter` не нужен.

Java Consumer — базовый

Определения:

- `KafkaConsumer<K, V>` — класс consumer: подписывается на топики и получает записи.
- `poll(Duration timeout)` — запрос данных: возвращает **сразу**, если данные есть, иначе ждёт до `timeout` и возвращает **пустой** результат.
- `ConsumerRecord<K, V>` — полученная запись: `key()`, `value()`, `partition()`, `offset()`, `timestamp()`.
- `auto.offset.reset` — откуда читать, если у группы **нет закоммиченных offsets**: `earliest` — с начала, `latest` — только новые (по умолчанию), `none` — ошибка.



```
Map<String, Object> config = Map.of(
    ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092",
    ConsumerConfig.GROUP_ID_CONFIG, "my-java-application",
    ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest"
);

try (var consumer = new KafkaConsumer<>(config, new StringDeserializer(),
    new StringDeserializer())) {
    consumer.subscribe(List.of("demo_java")); // партии между
    консьюмерами группы распределит Kafka

    while (true) {
        var records = consumer.poll(Duration.ofMillis(1000));
        for (var record : records) {
```

```

        log.info("key={} value={} partition={} offset={}",
                record.key(), record.value(), record.partition(),
                record.offset());
    }
}

```

- обязательно: адрес брокера, `group.id`, десериализаторы;
- первый запуск с `earliest` читает **всё**; повторный запуск той же группы продолжает с **закоммиченного offset**;
- offsets коммитятся **автоматически** (`enable.auto.commit=true`, раз в 5 с внутри `poll()`);
- сообщение KIP-848 is production-ready при запуске — про новый протокол групп (`group.protocol=consumer`), см. [Rebalance](#).

Результат

Группа уже читала топик → старт с закоммиченных offsets, только новые сообщения:

```

... Adding newly assigned partitions: [demo_java-0, demo_java-1, demo_java-2]
... Setting offset for partition demo_java-0 to the committed offset
FetchPosition{offset=338 ...}
... Setting offset for partition demo_java-1 to the committed offset
FetchPosition{offset=124 ...}
... Setting offset for partition demo_java-2 to the committed offset
FetchPosition{offset=190 ...}
[main] INFO ConsumerDemo - key=id_1 value=hello world 1 partition=0
offset=338
...
[main] INFO ConsumerDemo - key=null value=hello world 29 partition=2
offset=260
[main] INFO ConsumerDemo - Polling
всё прочитано, ждёт новые

```

Партиция	Offsets	Сообщения	Producer
0	338–343	id_1, id_3, id_6 ×2	ProducerDemoKeys
0	344–493	key=null, hello world 0...29	ProducerDemoWithCallback
1	124–127	id_0, id_8 ×2	ProducerDemoKeys
1	128–217	key=null, hello world 0...29	ProducerDemoWithCallback
2	190	key=null, hello world	ProducerDemo

Партиция	Offsets	Сообщения	Producer
2	191–200	id_2 , id_4 , id_5 , id_7 , id_9 ×2	ProducerDemoKeys
2	201–260	key=null , hello world 0...29	ProducerDemoWithCallback

- записи приходят **блоками по партициям** (P0, потом P1, потом P2), внутри партиции — в порядке запуска producers;
- ключи легли как в ProducerDemoKeys : id_1/3/6 → P0, id_0/8 → P1, остальные → P2;
- у **новой** группы вместо committed offset будет Found no committed offset → earliest → с offset 0;
- перезапуск сразу после Stop может дать не все партиции: убитый экземпляр ещё ~45 с (session.timeout.ms) числится в группе → лечится graceful shutdown.

Graceful Shutdown

Определения:

- **Graceful shutdown** — корректная остановка consumer: дочитать текущий poll , закоммитить offsets и **выйти из группы**.
- **Shutdown hook** — поток, который JVM запускает при завершении процесса (Ctrl+C / SIGINT, SIGTERM, Stop в IDE): Runtime.getRuntime().addShutdownHook(...) .
- consumer.wakeup() — единственный метод KafkaConsumer , безопасный для вызова **из другого потока**: текущий или следующий poll() бросает WakeupException .

Без graceful shutdown процесс просто убит:

- последние offsets могут не закоммититься → после рестарта сообщения придут повторно;
- consumer не выходит из группы → координатор ждёт session.timeout.ms (45 с), его партиции всё это время никто не читает.

Алгоритм:

1. Сигнал → JVM запускает shutdown hook в отдельном потоке.
2. Hook вызывает consumer.wakeup() .
3. poll() в main бросает WakeupException → выход из while (true) .
4. consumer.close() → коммит offsets + LeaveGroup → rebalance **сразу**.
5. Hook ждёт mainThread.join() , чтобы JVM не завершилась раньше close() .

```

try (var consumer = new KafkaConsumer<>(config, new StringDeserializer(),
new StringDeserializer())) {

    Thread mainThread = Thread.currentThread();

    Runtime.getRuntime().addShutdownHook(new Thread(() -> {
        log.info("Detected a shutdown, let's exit by calling
consumer.wakeup()...");
        consumer.wakeup(); // close() отсюда нельзя:
KafkaConsumer не потокобезопасен
        try {
            mainThread.join(); // дождаться, пока main закроет
consumer
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }));

    consumer.subscribe(List.of("demo_java"));

    while (true) {
        var records = consumer.poll(Duration.ofMillis(1000));
        for (var record : records) {
            log.info("key={} value={} partition={} offset={}",
record.offset(),
                record.key(), record.value(), record.partition(),
record.offset());
        }
    }

} catch (WakeupException e) {
    log.info("Consumer is starting to shut down"); // ожидаемо;
consumer уже закрыт try-with-resources
} catch (Exception e) {
    log.error("Unexpected exception in the consumer", e);
}
log.info("The consumer is now gracefully shut down");

```

Результат (Stop в IntelliJ)

```

[main] INFO ConsumerDemoWithShutdown - Polling
[Thread-0] INFO ConsumerDemoWithShutdown - Detected a shutdown, let's exit
by calling consumer.wakeup()... ← hook
... Revoke previously assigned partitions [demo_java-0, demo_java-1,
demo_java-2] ← close(): отдаёт партиции
... sending LeaveGroup request to coordinator localhost:9092 ... due to the

```

```
consumer is being closed          ← выходит из группы
... App info kafka.consumer for consumer-my-java-application-1 unregistered
[main] INFO ConsumerDemoWithShutdown - Consumer is starting to shut down
← catch (WakeupException)
[main] INFO ConsumerDemoWithShutdown - The consumer is now gracefully shut
down
```

Process finished with exit code 130 (interrupted by signal 2:SIGINT)

- hook — в потоке Thread-0, остальное — в main;
- close() (Revoke → LeaveGroup → unregistered) отработал до catch: try-with-resources закрывает ресурс до входа в catch;
- exit code 130 = 128 + SIGINT (2); из терминала то же самое по Ctrl+C.

⚠ Hook срабатывает, только если программу запускает IntelliJ: **Settings** → **Build, Execution, Deployment** → **Build Tools** → **Gradle** → **Build and run using: IntelliJ IDEA**. При запуске через Gradle Stop отменяет задачу (Build cancelled while executing task), java-процесс убивается без SIGINT/SIGTERM — hook не срабатывает, consumer остаётся в группе «призраком».

Consumer Groups и rebalance

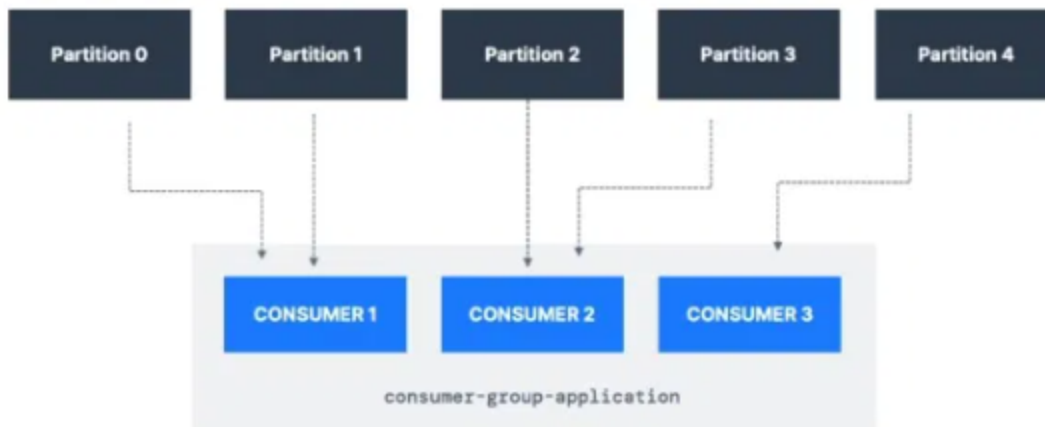
Consumer group — как группа делит топик

Определения:

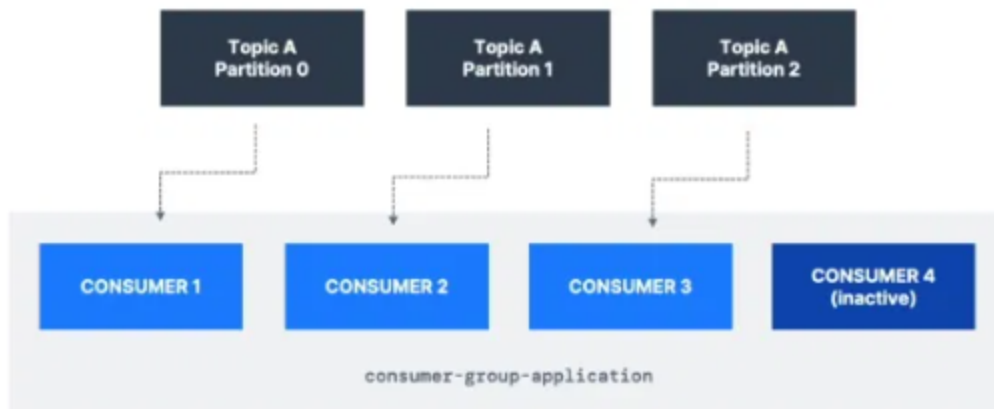
- **Consumer group** — консьюмеры с одинаковым group.id, которые вместе читают топик как одно приложение.
- **Group coordinator** — брокер, который распределяет партиции между участниками группы.
- **Rebalance** — перераспределение партиций при входе/выходе консьюмера или изменении числа партиций.

Правила распределения

- каждую партицию в группе читает **ровно один** консьюмер; один консьюмер может читать несколько;
- пример: 5 партиций, 3 консьюмера → C1: P0, P1 · C2: P2, P3 · C3: P4.

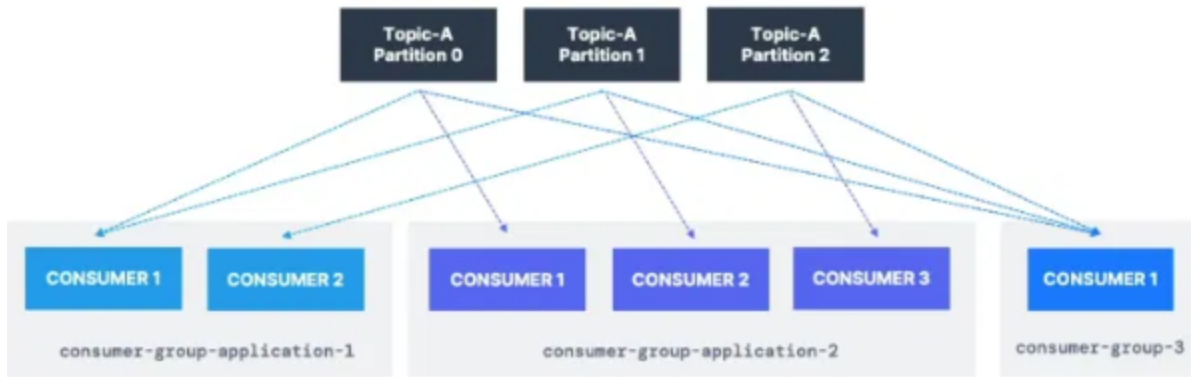


- консьюмеров **больше**, чем партиций → лишние **inactive** (3 партиции, 4 консьюмера → С4 простаивает);
- максимальный параллелизм группы = **число партиций**; простаивающий консьюмер — резерв при падении другого.



Несколько групп на одном топике

- каждая группа получает **все** сообщения топика, у каждой свои offsets;
- пример: group-1 (2 консьюмера), group-2 (3), group-3 (1 консьюмер читает все 3 партиции);
- типичный случай: orders читают billing-service и notification-service, у каждого свой group.id.



Группы на практике — CLI и Java

CLI: `--group`

```
# два консьюмера в ОДНОЙ группе → партии (и сообщения) делятся между ними
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic
first_topic --group my-first-application
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic
first_topic --group my-first-application

# ДРУГАЯ группа с начала → получает все сообщения
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic
first_topic --group my-second-application --from-beginning
```

- `--from-beginning` работает, **только пока у группы нет закоммиченных offsets**; повторный запуск продолжит с последнего offset;
- без `--group` создаётся случайная группа `console-consumer-XXXXX` → каждый запуск — новая группа;
- чтобы увидеть распределение, producer — с `RoundRobinPartitioner` (см. [kafka-console-producer.sh](#)).

CLI: `kafka-consumer-groups.sh`

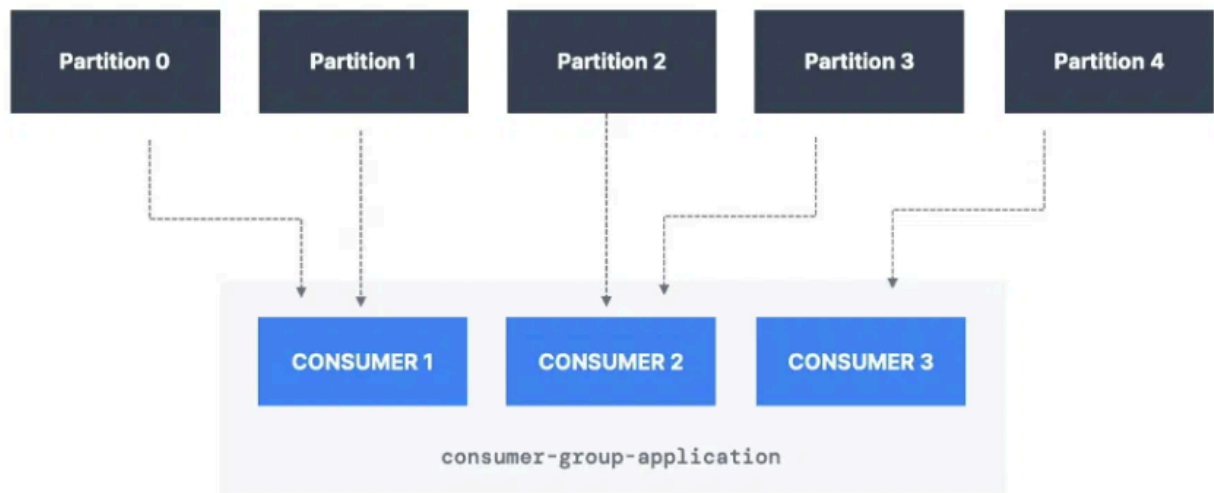
```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --list
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --
group my-first-application
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --
group my-first-application --members # участники
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --
group my-first-application --state # состояние
```

Колонка -- describe	Значение
PARTITION	партиция топика
CURRENT-OFFSET	закоммиченный offset группы
LOG-END-OFFSET	offset последнего сообщения в партиции
LAG	LOG-END-OFFSET - CURRENT-OFFSET — сколько ещё не прочитано
CONSUMER-ID , HOST	кто читает партицию (пусто, если консьюмеров нет)

Нет запущенных консьюмеров → Consumer group '...' has no active members , но offsets и lag показываються; после запуска консьюмера lag уходит в 0.

Java: несколько экземпляров

- тот же ConsumerDemoWithShutdown , все экземпляры с **одним** group.id ;
- второй экземпляр в IntelliJ: **Edit Configurations** → **Modify options** → **Allow multiple instances**;
- смотреть в логах Adding newly assigned partitions у каждого.



Консьюмеров (топик с 3 партициями)	Распределение (RangeAssignor)
1	C1: P0, P1, P2
2	C1: P0, P1 · C2: P2
3	C1: P0 · C2: P1 · C3: P2
4	по одной у трёх, четвёртый простаивает

```
... (Re-)joining group                                     ← y
старых консьюмеров при входе нового
... Revoke previously assigned partitions [demo_java-0, demo_java-1,
demo_java-2]
... Adding newly assigned partitions: [demo_java-0, demo_java-1]   ← C1
... Adding newly assigned partitions: [demo_java-2]               ← C2
... Setting offset for partition demo_java-2 to the committed offset
FetchPosition{offset=261 ...}
[main] INFO ConsumerDemoWithShutdown - key=id_2 value=hello world 2
partition=2 offset=261
[main] INFO ConsumerDemoWithShutdown - key=id_4 value=hello world 4
partition=2 offset=262
```

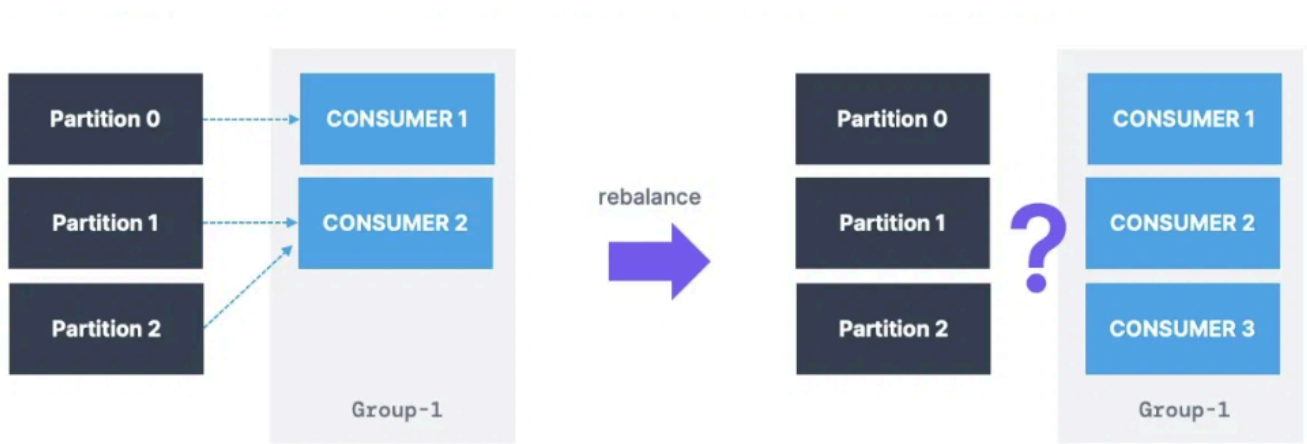
- C2 получил только P2 → читает только `id_2/4/5/7/9`; `id_1/3/6` (P0) и `id_0/8` (P1) ушли C1;
- каждое сообщение читает **один** консьюмер группы — владелец партиции;
- при каждом **rebalance** **все** отдают партиции (`Revoke`) и получают новые (`Adding`) — это **eager**;
- Stop с graceful shutdown → `LeaveGroup` → партиции сразу уходят оставшимся;
- Range делит партиции **каждого топика** по порядку: первые консьюмеры (по `member id`) получают на одну больше, если не делится поровну.

Rebalance — eager, cooperative, static membership

Определения:

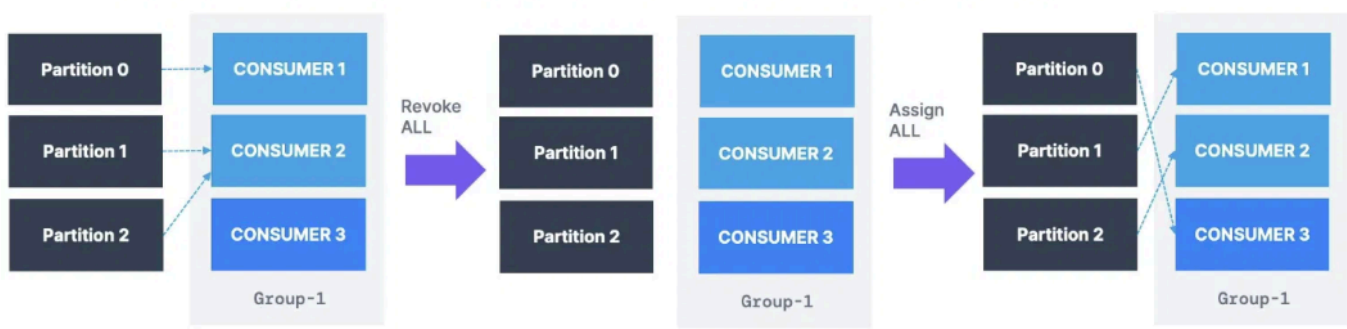
- **Eager rebalance** — все консьюмеры отдают **все** партиции, затем получают новое распределение.
- **Cooperative (incremental) rebalance** — переназначается только **часть** партиций, остальные консьюмеры читают без остановки.
- **Static membership** — консьюмер с постоянным `group.instance.id`, который может ненадолго уйти и вернуться **без rebalance**.
- **Partition assignor** — стратегия деления партиций (`partition.assignment.strategy`).

Когда rebalance: консьюмер входит/выходит, администратор добавляет партиции.

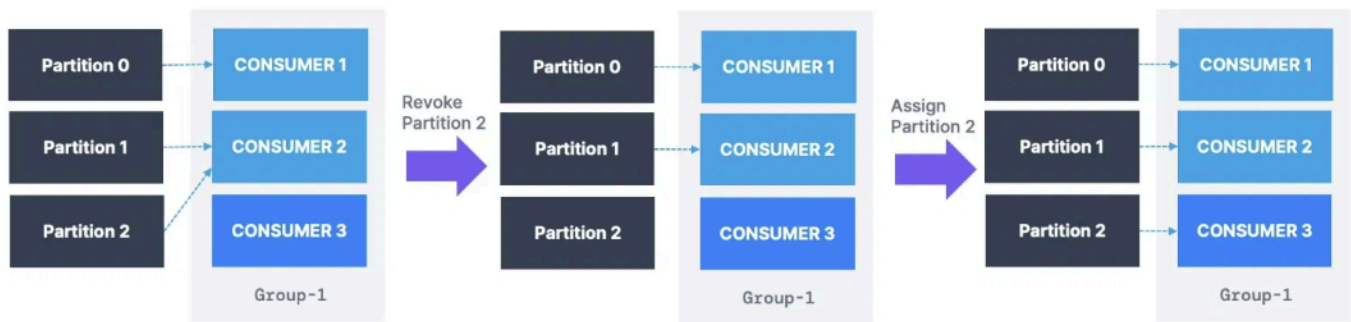


Eager vs Cooperative

	Eager	Cooperative
Что отдаётся	все партии у всех (Revoke ALL → Assign ALL)	небольшое подмножество от одного консьюмера к другому
Остальные консьюмеры	«stop the world»: вся группа не читает	продолжают читать свои партии
Те же партии после	не обязательно	незатронутые остаются
Итерации	одна	может быть несколько (отсюда «incremental»)
В логе	Revoke previously assigned partitions [все]	Revoked partitions (owned – assigned): [только нужные]



Пример cooperative: пришёл C3 → у C2 отбирается только P2 и отдаётся C3; P0 и P1 не трогаются.



Стратегии назначения

Assignor	Как делит	Rebalance
RangeAssignor	по каждому топике отдельно, возможен перекос	eager
RoundRobinAssignor	по кругу по всем топикам, оптимальный баланс	eager
StickyAssignor	как RoundRobin + минимум перемещений	eager
CooperativeStickyAssignor	как Sticky	cooperative

- по умолчанию **[RangeAssignor, CooperativeStickyAssignor]** → используется Range; на CooperativeSticky переходят одним rolling restart, убрав Range из списка;
- в группе выбирается протокол, который поддерживают **все** участники: старые [range, cooperative-sticky] + новый [cooperative-sticky] → договорятся на cooperative;
- Kafka Connect** — cooperative по умолчанию; **Kafka Streams** — через StreamsPartitionAssignor.

```
Map<String, Object> config = Map.of(
    ConsumerConfig.BootstrapServersConfig, "localhost:9092",
    ConsumerConfig.GroupIdConfig, "my-java-application",
    ConsumerConfig.AutoOffsetResetConfig, "earliest",
    ConsumerConfig.PartitionAssignmentStrategyConfig,
    CooperativeStickyAssignor.class.getName()
    // , ConsumerConfig.GroupInstanceIdConfig, "consumer-1" //
    static membership
);
```

Лог cooperative — видно, что добавилось и что отобрали:

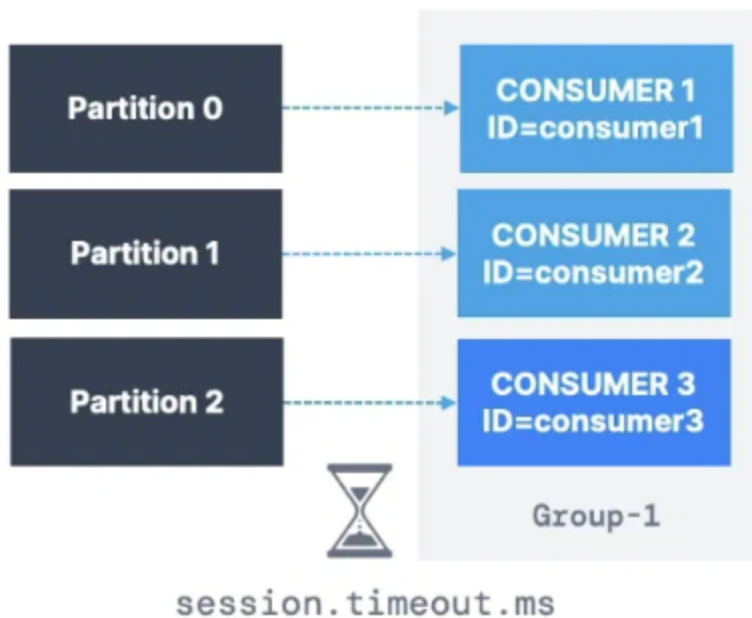
```
... Updating assignment with
Assigned partitions:           [demo_java-0, demo_java-1]
Current owned partitions:      [demo_java-0, demo_java-1,
```

```
demo_java-2]
  Added partitions (assigned - owned):      []
  Revoked partitions (owned - assigned):    [demo_java-2]      ←
отдал только P2
```

Rebalance может пройти в **2 этапа**: сначала партиция отбирается у старого владельца, затем отдаётся новому.

Static group membership

- по умолчанию вернувшийся консьюмер получает **новый member ID** и новые партиции;
- с `group.instance.id` он static member: есть `session.timeout.ms` (45 с), чтобы вернуться и получить **свои же** партиции **без rebalance**;
- при `close()` static member **не** шлёт `LeaveGroup` — группа ждёт его;
- id уникален для каждого экземпляра, дубль → `FencedInstanceIdException` ;
- полезно, когда консьюмер держит **локальное состояние** / кэш (рестарт пода).



Новый протокол групп — KIP-848

- `group.protocol=consumer` , production-ready с **Kafka 4.0**;
- распределение считает **брокер**, rebalance всегда инкрементальный, без синхронизации всей группы;
- `partition.assignment.strategy` не используется → серверный `group.remote.assignor (uniform / range)`;
- в курсе разбирается классический протокол.

Offsets и семантики доставки

Consumer offsets — где группа остановилась

Определение: **committed offset** — сохранённая позиция, до которой группа обработала партицию. Хранится отдельно для каждой пары «группа + партиция» во внутреннем топике `__consumer_offsets`.



- consumer периодически **коммитит** offsets; запись в `__consumer_offsets` делает брокер;
- коммитится offset **следующего** сообщения: последний обработанный + 1;
- consumer упал → группа продолжит с **последнего закоммиченного offset**;
- у новой группы offsets нет → старт по `auto.offset.reset` (`latest` по умолчанию / `earliest`).

Коммит offsets — авто и ручной

Определения:

- `enable.auto.commit` — consumer сам периодически коммитит offsets (по умолчанию `true`).
- `auto.commit.interval.ms` — интервал авто-коммита (по умолчанию `5000` мс).
- `commitSync()` / `commitAsync()` — ручной коммит: ждёт ответа брокера / не ждёт.

Как работает авто-коммит

- коммит происходит **внутри** `poll()` , если с прошлого коммита прошло `auto.commit.interval.ms` ;
- коммитятся offsets записей, которые вернул **предыдущий** `poll()` ;
- также коммитит при `close()` и перед отдачей партиций при `rebalance` → без `graceful shutdown` последние ~5 с прочитанного придут повторно.

poll() → старт таймера
poll() → прошло 3 с — коммита нет
poll() → прошло 6 с — внутри poll() вызывается commitAsync()



Условие at least once

Все записи из `poll()` должны быть обработаны до следующего `poll()`. Иначе (например, асинхронная обработка в других потоках) следующий `poll()` закоммитит необработанное, и при падении оно потеряется. Тогда: `enable.auto.commit=false` + ручной коммит правильных offsets.

```
Map<String, Object> config = Map.of(
    // ...
    ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false
);

while (true) {
    var records = consumer.poll(Duration.ofMillis(1000));
    for (var record : records) {
        process(record); // сначала обработали
    }
    consumer.commitSync(); // потом закоммитили → at least once
}
```

Сброс offsets — reset-offsets

Определение: `kafka-consumer-groups.sh --reset-offsets` переставляет закоммиченные `offsets` группы — чтобы перечитать данные или пропустить часть.

```
# dry run: только показывает новые offsets
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --group my-first-application --reset-offsets --to-earliest --topic first_topic

# применить
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --group my-first-application --reset-offsets --to-earliest --execute --topic first_topic

# перечитать 2 последних сообщения в КАЖДОЙ партии
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --group my-first-application --reset-offsets --shift-by -2 --execute --topic first_topic
```

Флаг	Куда ставит offset
<code>--to-earliest</code>	начало партии
<code>--to-latest</code>	конец (пропустить старое)
<code>--to-offset 5</code>	конкретный offset
<code>--shift-by N</code>	сдвиг на N (+ вперёд, – назад)
<code>--to-datetime 2026-10-09T12:00:00.000</code>	первое сообщение после момента
<code>--by-duration PT1H</code>	час назад от текущего времени

- обязательна **область**: `--topic <t>` или `--all-topics`;
- без `--execute` — **dry run**;
- сдвиг применяется к **каждой партии**: 3 партии × `--shift-by -2` = до 6 сообщений;
- группа должна быть **неактивна** (все консьюмеры остановлены), иначе ошибка о состоянии `Stable`.

Delivery semantics — at most, at least, exactly once

Определение: **delivery semantics** — гарантия того, сколько раз сообщение повлияет на результат обработки. Определяется моментом коммита `offset` относительно обработки.

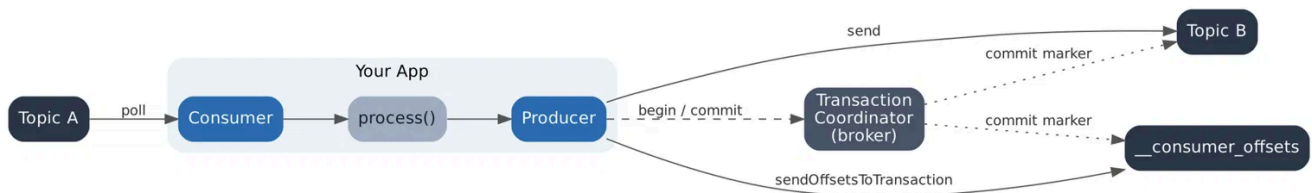
Семантика	Коммит	При сбое	Требование
At most once	сразу при получении, до обработки	сообщения теряются	—
At least once (обычно выбирают)	после обработки	сообщение читается снова → дубли	идемпотентная обработка
Exactly once	атомарно с результатом	эффект не задваивается	транзакции или идемпотентный приёмник

Java-consumer по умолчанию (авто-коммит) даёт **at least once** — при условии выше.

Exactly once: Kafka → Kafka — Transactional API

Сценарий **read-process-write**: читаем из топика A, пишем результат в B.

- **проблема**: запись в B и коммит offset в A — две операции; упали между ними → результат в B удвоится;
- **решение**: транзакция — запись в B и коммит offset в A **атомарно**;
- упали посреди транзакции → **abort**: offset не закоммичен, сообщение перечитывается, а уже записанный результат помечен отменённым и не виден консьюмерам с `isolation.level=read_committed`.



```
// producer: transactional.id=app-1
// consumer: enable.auto.commit=false, isolation.level=read_committed
producer.initTransactions();

while (true) {
    var records = consumer.poll(Duration.ofMillis(500));
    producer.beginTransaction();
    try {
        for (var r : records)
            producer.send(new ProducerRecord<>("topic-B", r.key(),
process(r.value())));
    }
}
```

```

    producer.sendOffsetsToTransaction(offsetsOf(records),
consumer.groupMetadata()); // offset в той же транзакции
    producer.commitTransaction();
} catch (KafkaException e) {
    producer.abortTransaction(); // откат, сообщения прочитаются снова
}
}

```

В **Kafka Streams** цикл не нужен: `processing.guarantee=exactly_once_v2`.

Exactly once: Kafka → внешняя система — идемпотентный consumer

Чужую БД / API Kafka в транзакцию не включит → **at least once + идемпотентная обработка**:

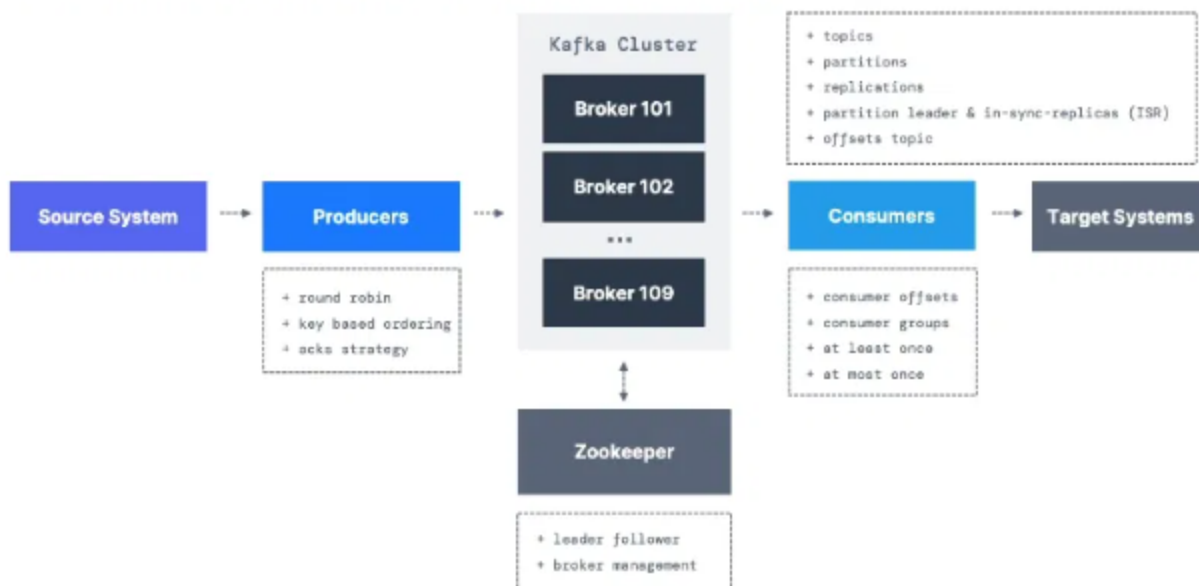
- запись по уникальному ID (бизнес-ID события или `topic + partition + offset`), дубль игнорируется или перезаписывается (upsert);
- или offset хранится **в той же БД и транзакции**, что и результат; при старте читается оттуда → `consumer.seek()`.

```

INSERT INTO payments (event_id, amount) VALUES (?, ?)
ON CONFLICT (event_id) DO NOTHING; -- повтор не создаст второй платёж

```

Итоговая схема



Блок	Ключевое
Producers	без ключа — round robin (сейчас sticky); с ключом — порядок по ключу; acks=0 / 1 / all
Cluster	topics → partitions с offsets; репликация (RF); leader + ISR; __consumer_offsets
Consumers	consumer groups делят партии; offsets группы; at least once (коммит после) / at most once (коммит до); exactly once — транзакции или идемпотентность
Координация	Zookeeper (leader/follower, управление брокерами) → в Kafka 4.x только KRaft

Вопросы к собеседованию

Общее и архитектура

Что такое Apache Kafka?

Это распределённая система с открытым исходным кодом, разработанная для высокоскоростной передачи больших объёмов данных с минимальной задержкой.

Преимущества

- Персистентность данных
- Высокая производительность
- Независимость пайплайнов обработки
- Возможность просмотреть историю записей заново
- Гибкость в использовании

Когда использовать

- λ -архитектура или k-архитектура
- Стриминг больших данных
- Много клиентов (producer и consumer)
- Требуется кратное масштабирование

Чего в Kafka нет из коробки

- Это не брокер сообщений

- Отложенные сообщения
- DLQ
- AMQP / MQTT
- TTL на сообщение
- Очереди с приоритетами

Основные компоненты Kafka

- **Producer (Производитель)** — приложение, которое публикует сообщения в топики Kafka
- **Consumer (Потребитель)** — приложение, которое подписывается на топики и читает сообщения
- **Broker (Брокер)** — сервер Kafka, который принимает, хранит и распределяет сообщения. В кластере Kafka может быть несколько брокеров
- **Topic (Топик)** — логическое разделение, по которому организуются данные. Производители отправляют сообщения в топики, а потребители читают из них
- **Partition (Раздел)** — каждый топик разделён на партиции для параллельной обработки. Сообщения в партициях упорядочены
- **Zookeeper** — сервис, используемый Kafka для управления состоянием кластера и координации брокеров.
Однако в новых версиях Kafka отказывается от Zookeeper в пользу собственного механизма метаданных KRaft (Kafka Raft).
Это новая внутренняя архитектура метаданных Kafka, которая устраняет зависимость от Zookeeper. Она основана на Raft-консенсусе, позволяя Kafka брокерам самостоятельно управлять метаданными и координировать взаимодействие между собой.

Чем controller отличается от брокера в KRaft?

Брокер хранит партиции с сообщениями и обслуживает клиентов (producer, consumer).

Controller хранит только метаданные кластера (брокеры, топики, партиции, лидеры, ISR) в топики `__cluster_metadata` и принимает решения: выбор лидеров партиций, создание топиков, реакция на падение брокера.

- контроллеры образуют кворум по Raft: решения принимает quorum leader, остальные — горячий резерв;
- в проде это отдельные узлы (`process.roles=controller / broker`), в разработке один узел может совмещать роли.

Зачем KRaft вместо Zookeeper?

Одна система вместо двух, масштаб до миллионов партиций (Zookeeper упирался примерно в 100 000), быстрее failover контроллера, единая модель безопасности. KRaft production-ready с Kafka 3.3, в Kafka 4.x Zookeeper удалён.

Топики, партиции и хранение

Архитектура топика

- **Топик разбит на партиции** — сообщения в топике распределяются по партициям для более эффективной параллельной обработки и хранения
- **Партиции хранятся на диске** — Kafka сохраняет данные на диск, что позволяет долговременно хранить сообщения
- **Партиции делятся на сегменты** — сегмент представляет собой обычный файл на диске, сегменты делятся на пассивные и активный. Запись происходит в активный сегмент
- **Данные удаляются либо по времени, либо по размеру**. Удаление происходит посегментно, с самого старого сегмента
 - **retention.bytes** - по максимальному размеру
 - **retention.ms** - по времени
- **Сообщение можно быстро найти по его Offset** — каждому сообщению в партиции присваивается уникальный смещающий индекс (offset), по которому можно легко найти сообщение

Настройки топика Kafka

Репликация

- `replication.factor`
 - **Описание:** Количество реплик для каждой партиции топика
 - **Пример:** `replication.factor=3`
- `min.insync.replicas`
 - **Описание:** Минимальное количество синхронизированных реплик
 - **Пример:** `min.insync.replicas=2`

Хранение данных

- `retention.ms`
 - **Описание:** Время хранения сообщений в топике в миллисекундах
 - **Пример:** `retention.ms=604800000` (7 дней)

- `retention.bytes`
 - **Описание:** Максимальный объём данных в топике, после чего старые сообщения удаляются
 - **Пример:** `retention.bytes=10737418240` (10 GB)
- `segment.bytes`
 - **Описание:** Размер сегмента логов топика
 - **Пример:** `segment.bytes=1073741824` (1 GB)

Политики очистки

- `cleanup.policy`
 - **Описание:** Как Kafka обрабатывает старые сообщения
 - **Значения:** `delete`, `compact`
 - **Пример:** `cleanup.policy=delete`

Партиции

- `num.partitions`
 - **Описание:** Количество партиций в топике
 - **Пример:** `num.partitions=3`

Для чего нужен Broker log cleaner thread?

Поток очистки журнала в Kafka отвечает за выполнение сжатия журнала. Сжатие журнала - это механизм, при котором Kafka удаляет избыточные записи, сохраняя только последнее значение для каждого ключа. Это полезно в тех случаях, когда требуется только последнее обновление для данного ключа, например, для обслуживания changelog или состояния БД. Программа очистки журналов периодически запускается для сжатия соответствующих партиций.

Чем топик отличается от партиции?

Топик — логический именованный поток, существует как запись в метаданных (имя, ID, число партиций, настройки). Партиция — его физическая часть: append-only лог на брокере. Порядок и offset существуют только внутри партиции.

Как гарантировать порядок сообщений?

Порядок есть только внутри партиции → связанным сообщениям дают общий ключ: `hash(key) % numPartitions` кладёт их в одну партицию. Число партиций после этого не

меняют. Чтобы ретраи не переставляли сообщения, нужен `enable.idempotence=true` (по умолчанию с Kafka 3.0).

Что будет при увеличении числа партиций?

Изменится результат `% numPartitions` — ключи начнут попадать в другие партиции, порядок по ключу нарушится. Уменьшить число партиций нельзя.

Как партиция хранится на диске?

- каждая реплика партиции — каталог `<топик>--<номер>` в `log.dirs` брокера;
- каталог делится на **сегменты**; новый сегмент создаётся по `segment.bytes` (1 ГБ) или `segment.ms` (7 дней) — что наступит раньше; писать можно только в последний, активный;
- сегмент = `.log` (батчи сообщений) + `.index` (`offset` → позиция в байтах, разреженный, ~раз в 4 КБ) + `.timeindex` (`timestamp` → `offset`);
- имя файлов — `base offset` сегмента (20 цифр);
- поиск `offset`: сегмент по имени файла → ближайшая запись в `.index` → дочитать `.log`.

Как работает retention и чем он отличается от compaction?

- `cleanup.policy=delete` (по умолчанию): удаляются **закрытые сегменты целиком**, когда самое новое сообщение в сегменте старше `retention.ms` (7 дней) или партиция больше `retention.bytes` (по умолчанию без лимита). Активный сегмент не удаляется никогда;
- `cleanup.policy=compact`: `log cleaner` оставляет для каждого ключа только последнее значение; сообщение с `value = null` (tombstone) удаляет ключ. Так хранится `__consumer_offsets`.

Брокеры и репликация

Архитектура брокера

- **У каждой партиции свой лидер** — в Kafka для каждой партиции в топике назначается лидер-брокер, который отвечает за запись и чтение данных
- **Сообщения пишутся в лидера** — производители отправляют сообщения напрямую в брокер-лидер партиции

- **Данные реплицируются между брокерами** — для обеспечения отказоустойчивости Kafka реплицирует данные партиций на другие брокеры, которые становятся репликами
- **Автоматический фейловер лидера** — в случае сбоя брокера-лидера Kafka автоматически назначает новый лидер из числа реплик, обеспечивая бесшовную работу системы

Настройки брокера Kafka

Репликация и консистентность

- `min.insync.replicas`
 - **Описание:** Минимальное количество синхронизированных реплик для подтверждения записи
 - **Пример:** `min.insync.replicas=2`
- `unclean.leader.election.enable`
 - **Описание:** Разрешает выбор лидера из неактуальных реплик, если нет синхронизированных реплик
 - **Пример:** `unclean.leader.election.enable=false`

Логирование и хранение данных

- `log.dirs`
 - **Описание:** Директория на диске, где хранятся логи партиций
 - **Пример:** `log.dirs=/var/lib/kafka/logs`
- `log.retention.hours`
 - **Описание:** Максимальное время хранения данных в логах
 - **Пример:** `log.retention.hours=168` (7 дней)
- `log.segment.bytes`
 - **Описание:** Максимальный размер сегмента лога, после чего создаётся новый
 - **Пример:** `log.segment.bytes=1073741824` (1 GB)

Производительность и задержки

- `num.network.threads`
 - **Описание:** Количество потоков для обработки сетевых запросов
 - **Пример:** `num.network.threads=3`
- `num.io.threads`
 - **Описание:** Количество потоков для ввода-вывода
 - **Пример:** `num.io.threads=8`
- `socket.send.buffer.bytes`

- **Описание:** Размер буфера для отправки данных по сети
- **Пример:** `socket.send.buffer.bytes=102400`

Управление сообщениями

- `message.max.bytes`
 - **Описание:** Максимальный размер сообщения, которое брокер может принять
 - **Пример:** `message.max.bytes=1048576` (1 MB)
- `replica.fetch.max.bytes`
 - **Описание:** Максимальный размер данных для запроса реплики
 - **Пример:** `replica.fetch.max.bytes=1048576` (1 MB)

Безопасность

- `ssl.keystore.location`
 - **Описание:** Путь к хранилищу ключей SSL
 - **Пример:** `ssl.keystore.location=/var/private/ssl/kafka.keystore.jks`
- `ssl.truststore.location`
 - **Описание:** Путь к хранилищу доверенных сертификатов
 - **Пример:** `ssl.truststore.location=/var/private/ssl/kafka.truststore.jks`

Что такое ISR и как связаны `acks=all` и `min.insync.replicas`?

ISR — реплики, не отстающие от лидера (не дольше `replica.lag.time.max.ms`, 30 с). `acks=all` ждёт все **текущие** ISR; если ISR сжался до лидера, это фактически `acks=1`. `min.insync.replicas` запрещает запись при слишком маленьком ISR (`NotEnoughReplicasException`). Типично: `RF=3`, `acks=all`, `min.insync.replicas=2`.

Как выбирается и распределяется лидер партиии?

- у каждой партиии ровно один лидер → лидеров у топика столько, сколько партиций;
- при создании топика реплики раскладываются по брокерам со сдвигом, первая в списке — **preferred leader** → лидеры распределены равномерно; две реплики одной партиии на одном брокере не лежат никогда;
- лидер упал → контроллер выбирает нового из ISR; брокер вернулся → `auto.leader.rebalance.enable=true` возвращает лидерство preferred leader.

Producer

Архитектура продюсера

- **Создание сообщения (Record):** Продюсер формирует сообщение, содержащее ключ (необязательный), значение и метаданные, такие как время отправки. Сообщение отправляется в топик (Topic), который состоит из одной или нескольких партиций
- **Выбор партиции:** Если ключ сообщения указан, Kafka использует его для хеширования и определения, в какую партицию записать сообщение (сообщения с одинаковым ключом попадают в одну и ту же партицию). Если ключа нет, Kafka распределяет сообщения по партициям с помощью round-robin или по другим правилам
- **Отправка сообщений в буфер (Batching):** Для повышения производительности продюсер Kafka не отправляет каждое сообщение по отдельности, а группирует несколько сообщений в пакеты (batching), прежде чем отправить их брокеру. Это снижает сетевые задержки и нагрузку на брокера
- **Сжатие (Compression):** Для уменьшения объема передаваемых данных продюсер может сжимать сообщения с использованием таких алгоритмов, как GZIP, Snappy или LZ4. Сжатие снижает нагрузку на сеть и хранение, но добавляет небольшие накладные расходы на процессор
- **Асинхронная отправка:** Продюсер отправляет пакеты сообщений асинхронно. Это означает, что сообщения записываются в буфер памяти и отправляются брокеру, не ожидая завершения предыдущих операций. Это повышает пропускную способность
- **Подтверждения (Acknowledgments):** Kafka позволяет настраивать уровень подтверждений от брокеров
- **Ретрай и идемпотентность:** Если отправка сообщения не удалась, продюсер может повторить попытку отправки (ретрай). Также можно включить идемпотентный режим продюсера, что предотвращает повторную отправку одного и того же сообщения в случае сбоя, обеспечивая отправку уникального сообщения один раз
- **Error handling:** Продюсер обрабатывает ошибки при отправке сообщений. В зависимости от настроек продюсер может попытаться переотправить сообщение или сообщить о проблеме через callback

Резюме

- Продюсер выбирает партицию для сообщения
- Продюсер выбирает уровень гарантии доставки
- В продюсере можно тюнить производительность

Настройки продюсера

Bootstrap-серверы (`bootstrap.servers`)

- **Описание:** Указывает адреса брокеров Kafka, к которым продюсер должен подключаться для отправки сообщений
- **Пример:** `bootstrap.servers: localhost:9092,localhost:9093`
- **Зачем это нужно:** Kafka продюсер использует эти брокеры для получения метаданных о кластере (например, информация о топиках и партициях). Эти брокеры служат точками входа в кластер Kafka.

Сериализация ключа и значения

Продюсер должен преобразовывать (сериализовать) данные в байтовый формат перед отправкой в Kafka

- **Ключевая настройка для сериализации ключа:**
 - `key.serializer`
 - Пример: `key.serializer: org.apache.kafka.common.serialization.StringSerializer`
- **Ключевая настройка для сериализации значения:**
 - `value.serializer`
 - Пример: `value.serializer: org.apache.kafka.common.serialization.StringSerializer`

Варианты сериализаторов:

- `StringSerializer` для строк
- `ByteArraySerializer` для массива байтов
- `LongSerializer` для чисел
- Также можно реализовать свои собственные сериализаторы

Отправка сообщений в буфер

Продюсер Kafka отправляет сообщения асинхронно, и для этого используется буферизация сообщений

- **`batch.size`:** Размер одного пакета (batch), который продюсер отправляет брокеру
 - **Описание:** Определяет количество байтов сообщений, которые могут быть буферизованы в одном пакете перед отправкой брокеру
 - **Пример:** `"batch.size": 16384` (16 KB)

- **Зачем это нужно:** Большие пакеты могут повысить производительность, но могут увеличить задержки
- **linger.ms:** Максимальное время ожидания перед отправкой пакета
 - **Описание:** Продюсер может немного подождать, пока буфер накопит сообщения, чтобы отправить больше данных за один раз
 - **Пример:** `linger.ms: 5` (время ожидания 5 мс)
 - **Зачем это нужно:** Позволяет продюсеру собирать больше сообщений в пакете перед отправкой, что может улучшить эффективность использования сети
- **buffer.memory:** Размер выделенной памяти для буферизации сообщений
 - **Описание:** Общий объем памяти, который продюсер может использовать для хранения сообщений, ожидающих отправки
 - **Пример:** `buffer.memory: 33554432` (32 MB)
 - **Зачем это нужно:** Если буфер заполняется, продюсер приостанавливает отправку сообщений, пока буфер не освободится

Сжатие сообщений

Продюсер может сжимать сообщения для уменьшения объема передаваемых данных

- **compression.type**
 - **Описание:** Указывает тип сжатия для сообщений
 - **Пример:** `compression.type: gzip` (варианты: none, gzip, snappy, lz4, zstd)
 - **Зачем это нужно:** Сжатие уменьшает объем данных, передаваемых по сети, что может снизить нагрузку на сеть и хранилище, особенно при больших объемах сообщений. Однако это может потребовать дополнительных ресурсов на сжатие/разжатие

Распределение сообщений по партициям (партиционирование)

- **partitioner.class**
 - **Описание:** определяет логику, по которой продюсер выбирает партицию для каждого сообщения
 - **Примеры:**
 - **если настройка не задана**, по умолчанию используется `DefaultPartitioner`, который может распределять сообщения по партициям равномерно или на основе ключа сообщения
 - `partitioner.class: o.a.k.clients.producer.RoundRobinPartitioner` использует метод Round Robin для распределения сообщений

- `partitioner.class: o.a.k.clients.producer.UniformStickyPartitioner`
равномерно отправляет сообщения, привязываясь к партиции на короткий промежуток времени, чтобы уменьшить нагрузку на брокеры

Подтверждения (acks)

Настройка определяет, как много брокеров должны подтвердить получение сообщения перед тем, как продюсер будет считать его успешно отправленным

- **acks**
 - **Описание:** Определяет количество подтверждений от брокеров
 - **Значения:**
 - `0` : Продюсер не ждёт подтверждений (самая быстрая отправка, но высокий риск потери сообщений)
 - `1` : Продюсер ждёт подтверждения от лидера партиции
 - `all` (или `-1`) : Продюсер ждёт подтверждений от всех реплик (наибольшая надежность, но увеличенные задержки)
 - **Пример:** `acks: all`
 - **Зачем это нужно:** Позволяет выбрать баланс между скоростью и надежностью отправки данных.

Дополнительные важные настройки

- **Количество повторных попыток (retries):**
 - **Описание:** Определяет, сколько раз продюсер должен попытаться отправить сообщение при неудаче
 - **Пример:** `retries: 3`
 - **Зачем это нужно:** Если произошёл временный сбой, продюсер может попытаться повторить отправку сообщений, что увеличивает шанс доставки
- **Идемпотентность продюсера (enable.idempotence):**
 - **Описание:** Включение идемпотентного режима, что предотвращает дублирование сообщений при сбоях
 - **Пример:** `enable.idempotence: true`
 - **Зачем это нужно:** Гарантирует, что каждое сообщение будет доставлено ровно один раз
- **Максимальный размер сообщения (max.request.size):**

- **Описание:** Максимальный размер сообщения, которое продюсер может отправить брокеру
- **Пример:** `max.request.size: 1048576` (1 MB)
- **Зачем это нужно:** Ограничивает размер сообщений, которые могут быть отправлены, чтобы избежать перегрузки сети и брокеров.
- **Таймаут ожидания подтверждений (`request.timeout.ms`):**
 - **Описание:** Максимальное время ожидания подтверждения от брокера
 - **Пример:** `request.timeout.ms: 30000` (30 секунд)
 - **Зачем это нужно:** Помогает избежать бесконечного ожидания ответа от брокера в случае его сбоя

Пример конфигурации Kafka Producer

```
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerRecord;
import java.util.Properties;

public class KafkaStringArrayProducer {

    public static void main(String[] args) {
        // Настройки Kafka Producer
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("key.serializer",
"org.apache.kafka.common.serialization.StringSerializer");
        props.put("value.serializer",
"org.apache.kafka.common.serialization.StringSerializer");

        // Создание Kafka Producer
        KafkaProducer<String, String[]> producer = new KafkaProducer<>
(props);

        String key = "user123";
        String[] value = {"message1", "message2", "message3"};

        // Создание записи и добавление заголовков
        ProducerRecord<String, String> record = new ProducerRecord<>
("my_topic", key, value);
        record.headers().add("traceId", "someTraceId");

        // Отправка сообщения в Kafka
        producer.send(record, (metadata, exception) -> {
            if (exception != null) {
                System.out.println("Ошибка при отправке сообщения: " +
```

```

exception.getMessage());
        } else {
            System.out.println("Сообщение отправлено в топик " +
metadata.topic() + " с партицией " + metadata.partition());
        }
    });

    producer.close();
}
}

```

```

acks=all
retries=3
compression.type=gzip

```

С использованием Spring Kafka

```

import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.common.serialization.StringSerializer;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.kafka.core.DefaultKafkaProducerFactory;
import org.springframework.kafka.core.ProducerFactory;
import org.springframework.kafka.config.ConcurrentMessageListenerContainer;
import org.springframework.kafka.listener.MessageListenerContainer;
import org.springframework.kafka.producer.Producer;
import org.springframework.kafka.producer.ProducerRecord;

import java.util.HashMap;
import java.util.Map;

@EnableKafka
@Configuration
public class KafkaProducerConfig {

    @Autowired
    private KafkaProperties kafkaProperties;

    @Bean
    public Map<String, Object> producerConfigs() {
        Map<String, Object> props = new HashMap<>();
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
kafkaProperties.getServer());
    }
}

```

```

        props.put(ProducerConfig.CLIENT_ID_CONFIG,
kafkaProperties.getProducerId());
        props.put(
            ProducerConfig.INTERCEPTOR_CLASSES_CONFIG,

"com.example.configuration.kafka.KafkaProducerLoggingInterceptor"
        );

        if ("SASL_SSL".equals(kafkaProperties.getSecurityProtocol())) {
            props.put("ssl.truststore.location",
kafkaProperties.getSslTrustStoreLocation());
            props.put("ssl.truststore.password",
kafkaProperties.getSslTrustStorePassword());
            props.put("ssl.truststore.type",
kafkaProperties.getSslTrustStoreType());
            props.put("ssl.keystore.type",
kafkaProperties.getSslKeyStoreType());

            props.put("sasl.mechanism", kafkaProperties.getSaslMechanism());
            props.put("security.protocol",
kafkaProperties.getSecurityProtocol());
            props.put("sasl.jaas.config",
kafkaProperties.getJaasConfigCompiled());
        }

        return props;
    }

    @Bean
    public ProducerFactory<String, String> producerFactory() {
        var stringSerializerKey = new StringSerializer();
        stringSerializerKey.configure(Map.of("key.serializer.encoding",
"UTF-8"), true);
        stringSerializerKey.configure(Map.of("serializer.encoding", "UTF-
8"), true);

        var stringSerializerValue = new StringSerializer();
        stringSerializerValue.configure(Map.of("value.serializer.encoding",
"UTF-8"), false);
        stringSerializerValue.configure(Map.of("serializer.encoding", "UTF-
8"), false);

        return new DefaultKafkaProducerFactory<>(producerConfigs(),
stringSerializerKey, stringSerializerValue);
    }

```

```

@Bean
public KafkaTemplate<String, String> kafkaTemplate() {
    return new KafkaTemplate<>(producerFactory());
}
}

```

```

import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Service;

@Service
public class KafkaProducerService {

    private final KafkaTemplate<String, String> kafkaTemplate;

    public KafkaProducerService(KafkaTemplate<String, String> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    public void sendMessage(String message, String key, String topic) {
        try {
            log.info("Sending message {}", data);
            kafkaTemplate.send(topic, key, message);
            log.info("Successfully send message {}", data);
        } catch (Exception ex) {
            log.error("Failed send message to {} topic by key {}", key, topic);
            throw ex;
        }
    }
}

```

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/kafka")
public class KafkaController {

    @Autowired
    private KafkaProducerService kafkaProducerService;

    @PostMapping("/send")
    public String sendMessage(@RequestParam String message, @RequestParam
String key, @RequestParam String topic) {

```

```

        kafkaProducerService.sendMessage(message, key, topic);
        return "Message sent to Kafka!";
    }
}

```

С использованием Spring Cloud Stream

```

spring:
  cloud:
    stream:
      bindings:
        output:
          destination: my_topic
      kafka:
        binder:
          brokers: localhost:9092

```

```

import org.springframework.cloud.stream.annotation.EnableBinding;
import org.springframework.cloud.stream.messaging.Source;
import org.springframework.integration.support.MessageBuilder;
import org.springframework.messaging.Message;
import org.springframework.stereotype.Service;

@Service
@EnableBinding(Source.class) // Подключение к каналу сообщений
public class KafkaStreamProducer {

    private final Source source;

    public KafkaStreamProducer(Source source) {
        this.source = source;
    }

    public void sendMessage(String message) {
        Message<String> msg = MessageBuilder.withPayload(message).build();
        source.output().send(msg); // Отправка сообщения в Kafka
    }
}

```

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/kafka-stream")

```

```
public class KafkaStreamController {

    @Autowired
    private KafkaStreamProducer kafkaStreamProducer;

    @PostMapping("/send")
    public String sendMessage(@RequestParam String message) {
        kafkaStreamProducer.sendMessage(message);
        return "Message sent to Kafka via Spring Cloud Stream!";
    }
}
```

Для чего нужен идемпотентный продюсер?

Идемпотентный продюсер гарантирует exactly-once гарантию доставки, предотвращая дублирование записей в Kafka в случае повторных попыток отправки сообщений. Это важно для поддержания целостности данных и правильности их обработки в системе, особенно в распределенных системах, где могут возникать ошибки связи или сбои.

Для чего нужен интерфейс Partitioner?

Интерфейс Partitioner в Producer API определяет в какую партицию топика будет отправлено сообщение. Partitioner по-умолчанию использует хэш ключа (если он присутствует) для выбора партиции, гарантируя, что сообщения с одним и тем же ключом всегда отправляются в одну и ту же партицию. Могут быть реализованы пользовательские Partitioner для управления распределением сообщений по партициям на основе определенной бизнес-логики или характеристик данных.

Как распределяются сообщения без ключа?

С Kafka 2.4 — **sticky partitioner**: producer заполняет батч для одной партиции и только потом переключается на другую. Меньше запросов к брокеру, на большом объёме распределение всё равно равномерное. Round robin — только если явно задать RoundRobinPartitioner.

Consumer и группы

Архитектура консюмера

Потребители используют **Kafka Consumer API** для взаимодействия с брокерами Kafka. Они получают сообщения и обрабатывают их согласно своей логике. Потребители могут быть объединены в группы **Consumer Groups**.

Резюме

- "Smart" консюмер
- Консюмер опрашивает кафку
- Консюмер отвечает за гарантию обработки
- Автоматические фейловер в консюмер-группе
- Независимая обработка разными консюмер-группе

Компоненты

Consumer Group

Kafka использует концепцию Consumer Groups, что позволяет нескольким потребителям работать вместе, чтобы параллельно обрабатывать данные из топиков. Каждый потребитель в группе обрабатывает только часть данных из топика, обеспечивая масштабируемость и балансировку нагрузки.

- Все сообщения из одного Kafka Topic делятся между всеми потребителями в группе
- Если в группе несколько потребителей, Kafka гарантирует, что каждая партиция топика будет обрабатываться только одним потребителем
- В случае если один из потребителей выходит из строя, его партиции автоматически перераспределяются между оставшимися активными потребителями

Offset (Смещение)

Потребитель отслеживает offset каждой партиции, чтобы понимать, с какого сообщения продолжать чтение. Смещение — это уникальный идентификатор каждого сообщения в партиции.

Потребители могут хранить offset в Kafka или вне её (например, в базе данных или файловой системе). Если потребитель отключается, он может возобновить обработку с того места, где остановился, прочитав сохранённый offset.

Poll (Опрос)

Потребители используют метод poll() для опроса Kafka на наличие новых сообщений. Это асинхронный процесс, и Kafka будет

отправлять потребителю доступные сообщения по мере их поступления.

- Потребитель может указывать тайм-аут, после которого метод `poll()` вернёт пустой результат, если сообщений нет.
- Потребитель должен обрабатывать сообщения, а затем снова опрашивать Kafka для получения новых данных.

Процесс работы

1. **Инициализация:** Потребитель подключается к Kafka-брокерам и присоединяется к `consumer group`. Он получает информацию о партиции топика, который будет читать.
2. **Подписка на топик:** Потребитель подписывается на определённые топика с помощью метода `subscribe()`.
3. **Опрос:** Потребитель вызывает метод `poll()` для получения новых сообщений. Если в очереди есть сообщения, они передаются потребителю для обработки.
4. **Обработка сообщений:** Потребитель обрабатывает сообщения, извлекая полезную информацию из каждого.
5. **Подтверждение обработки:** После обработки сообщения потребитель подтверждает обработку с помощью `commit()`.
Это обновляет **offset**, позволяя потребителю продолжить чтение с места, на котором остановился.
6. **Обработка ошибок:** В случае ошибки потребитель может решить, как повторить обработку сообщения (например, с использованием механизма повторных попыток).
7. **Завершение работы:** Когда потребитель завершает обработку, он выходит из `consumer group` и может закрыть соединение с Kafka.

Настройки консюмера

- **bootstrap.servers** — список брокеров, к которым будет подключаться потребитель
- **group.id** — идентификатор группы потребителей
- **auto.offset.reset** — настройка поведения при отсутствии `offset` (`earliest` для чтения с самого начала или `latest` для чтения с конца)
- **enable.auto.commit** — указывает, должен ли потребитель автоматически коммитить `offset`. Если `false`, потребитель должен делать это вручную
- **auto.commit.interval.ms** — определяет интервал времени между автоматическими коммитами `offset` сообщений, если включена автоматическая фиксация
- **max.poll.records** — максимальное количество сообщений, которые потребитель будет получать за один вызов `poll()`
- **session.timeout.ms** — максимальное время без общения с Kafka перед тем, как

потребитель считается недоступным

- **client.rack** — используется для указания серверной стойки или дата-центра. Это особенно важно в случае, если у вас есть распределённая инфраструктура Kafka с несколькими стойками или дата-центрами, где сообщения могут быть реплицированы между разными физическими местоположениями (например, несколькими дата-центрами).

Что такое Rack в контексте Kafka?

Rack — это метка, которая идентифицирует физическое местоположение брокеров Kafka. В Kafka можно задать rack для каждого брокера с помощью параметра `broker.rack`, чтобы управлять репликацией данных, предпочтительно размещая реплики на разных физических машинах или в разных дата-центрах.

Преимущества использования client.rack

- **Снижение задержек:** Kafka будет предпочитать, чтобы данные попадали в тот же rack, где находится клиент, что уменьшает время отклика
- **Повышенная отказоустойчивость:** С правильной настройкой `client.rack` и `broker.rack` можно улучшить отказоустойчивость за счет размещения реплик в разных физически удаленных местах
- **Лучшее использование ресурсов:** Правильное распределение нагрузки по rack помогает избежать перегрузки одного физического местоположения

Пример конфигурации Kafka Consumer

```
import org.apache.kafka.clients.consumer.ConsumerConfig;
import org.apache.kafka.clients.consumer.KafkaConsumer;
import org.apache.kafka.common.serialization.StringDeserializer;

import java.time.Duration;
import java.util.HashMap;
import java.util.Map;
import java.util.Collections;

public class KafkaConsumerExample {

    public static void main(String[] args) {
        String bootstrapServers = "localhost:9092";
        String groupId = "my-consumer-group";
        String topic = "my-topic";
```

```

// Настройки Consumer
Map<String, Object> consumerConfigs = new HashMap<>();
consumerConfigs.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
bootstrapServers);
consumerConfigs.put(ConsumerConfig.GROUP_ID_CONFIG, groupId);
consumerConfigs.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class);
consumerConfigs.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class);
consumerConfigs.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG,
"earliest");

// Создание Consumer
KafkaConsumer<String, String> consumer = new KafkaConsumer<>
(consumerConfigs);

// Подписка на тему
consumer.subscribe(Collections.singletonList(topic));

try {
    // Чтение сообщений из Kafka
    while (true) {
        var records = consumer.poll(Duration.ofSeconds(1));
        records.forEach(record -> System.out.println("Received
message: " + record.value()));
    }
} finally {
    consumer.close();
}
}
}

```

At least once

Чтобы гарантировать обработку сообщений хотя бы один раз, нужно коммитить после обработки.

```

import org.apache.kafka.clients.consumer.KafkaConsumer;
import org.apache.kafka.clients.consumer.ConsumerConfig;
import org.apache.kafka.common.serialization.StringDeserializer;

import java.time.Duration;
import java.util.HashMap;
import java.util.Map;
import java.util.Collections;

```

```

public class KafkaConsumerAtLeastOnce {

    public static void main(String[] args) {
        try {
            // Чтение сообщений
            while (true) {
                var records = consumer.poll(Duration.ofSeconds(1)); // Ожидание 1
секунду для получения сообщений
                process(records);
                consumer.commitAsync(); // Commit после обработки
            }
        } finally {
            consumer.close(); // Закрытие consumer
        }
    }
}

```

At most once

Чтобы гарантировать обработку сообщений не более одного раза, нужно коммитить до обработки или включить авто-подтверждение смещений

```
enable.auto.commit=true .
```

```

import org.apache.kafka.clients.consumer.KafkaConsumer;
import org.apache.kafka.clients.consumer.ConsumerConfig;
import org.apache.kafka.common.serialization.StringDeserializer;

import java.time.Duration;
import java.util.HashMap;
import java.util.Map;
import java.util.Collections;

public class KafkaConsumerAtLeastOnce {

    public static void main(String[] args) {
        try {
            // Чтение сообщений
            while (true) {
                var records = consumer.poll(Duration.ofSeconds(1)); // Ожидание 1
секунду для получения сообщений
                consumer.commitAsync(); // Commit перед обработкой
                process(records);
            }
        } finally {

```

```

        consumer.close(); // Закрытие consumer
    }
}
}

```

С использованием Spring Kafka

```

@EnableKafka
@Configuration
public class KafkaConsumerConfig {

    @Autowired
    private KafkaProperties kafkaProperties;

    @Bean
    public ConsumerFactory<String, String> consumerFactory() {
        return new DefaultKafkaConsumerFactory<>(consumerConfigs());
    }

    @Bean
    public Map<String, Object> consumerConfigs() {
        Map<String, Object> configs = new HashMap<>();
        configs.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
kafkaProperties.getServer());
        configs.put(ConsumerConfig.GROUP_ID_CONFIG,
kafkaProperties.getConsumerGroupId());
        configs.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class);
        configs.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class);
        return configs;
    }

    @Bean
    public
KafkaListenerContainerFactory<ConcurrentMessageListenerContainer<String,
String>> kafkaListenerContainerFactory() {
        ConcurrentMessageListenerContainerFactory<String, String> factory =
new ConcurrentMessageListenerContainerFactory<>();
        factory.setConsumerFactory(consumerFactory());
        return factory;
    }
}

```

```

import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.messaging.handler.annotation.Header;
import org.springframework.stereotype.Service;

@Service
public class KafkaConsumer {

    @KafkaListener(topics = "my_topic", groupId = "group_id")
    public void listen(@Payload String message,
                      @Header("traceId") String traceId,
                      @Header("correlationId") String correlationId) {
        System.out.println("Received message: " + message);
        System.out.println("Trace ID: " + traceId);
        System.out.println("Correlation ID: " + correlationId);
    }
}

```

At least once

```

spring:
  kafka:
    consumer:
      enable-auto-commit: false # Отключение авто-commit
      auto-offset-reset: earliest # Начинать чтение с самого начала (если
нет смещения)
      group-id: my-consumer-group
      max-poll-records: 500 # Максимальное количество сообщений для
обработки за один раз
      listener:
        ack-mode: manual # Ручное подтверждение

```

```

import org.apache.kafka.clients.consumer.ConsumerConfig;
import org.springframework.kafka.annotation.EnableKafka;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.listener.MessageListener;
import org.springframework.kafka.listener.MessageListenerContainer;
import
org.springframework.kafka.listener.config.DefaultMessageListenerContainer;
import org.springframework.kafka.core.ConsumerFactory;
import
org.springframework.kafka.listener.ConcurrentMessageListenerContainer;
import org.springframework.kafka.core.DefaultKafkaConsumerFactory;
import org.springframework.kafka.listener.MessageListener;
import org.springframework.kafka.listener.MessageListenerContainer;

```

```

@EnableKafka
public class AtLeastOnceConsumer {

    @KafkaListener(topics = "my-topic", groupId = "my-consumer-group")
    public void listen(String message, Acknowledgment acknowledgment) {
        System.out.println("Received message: " + message);
        // Обработка сообщения
        // Подтверждение смещения вручную после успешной обработки
        acknowledgment.acknowledge();
    }
}

```

At most once

```

spring:
  kafka:
    consumer:
      enable-auto-commit: true # Включение авто-commit
      group-id: my-consumer-group
      auto-offset-reset: earliest # Начинать чтение с самого начала
      max-poll-records: 100 # Максимальное количество сообщений для
        обработки за один раз

```

```

import org.springframework.kafka.annotation.KafkaListener;

public class AtMostOnceConsumer {

    @KafkaListener(topics = "my-topic", groupId = "my-consumer-group")
    public void listen(String message) {
        System.out.println("Received message: " + message);
        // Обработка сообщения...
        // Смещение будет автоматически зафиксировано после получения
        сообщения
    }
}

```

С использованием Spring Cloud Stream

```

spring:
  cloud:
    stream:
      bindings:
        input:

```

```
    destination: my-topic
    group: my-consumer-group
    content-type: application/json
kafka:
  binder:
    brokers: localhost:9092
    auto-create-topics: false
```

```
import org.springframework.cloud.stream.annotation.EnableBinding;
import org.springframework.cloud.stream.annotation.StreamListener;
import org.springframework.messaging.handler.annotation.Payload;
import org.springframework.stereotype.Service;

@Service
@EnableBinding(KafkaProcessor.class) // Указывает на интерфейс, с которым
// связывается этот сервис
public class KafkaConsumerService {

    // Метод будет слушать сообщения из указанного канала
    @StreamListener("input")
    public void handle(@Payload String message) {
        System.out.println("Received message: " + message);
    }
}
```

```
import org.springframework.cloud.stream.annotation.Input;
import org.springframework.messaging.SubscribableChannel;

public interface KafkaProcessor {

    @Input("input") // Имя канала, которое мы используем в application.yml
    SubscribableChannel input();
}
```

At least once

```
spring:
  cloud:
    stream:
      bindings:
        input:
          destination: my-topic
          group: my-consumer-group
          content-type: application/json
```

```
consumer:
  ackMode: manual # Ручное подтверждение
  maxAttempts: 3 # Максимальное количество попыток
```

```
import org.springframework.cloud.stream.annotation.EnableBinding;
import org.springframework.cloud.stream.annotation.StreamListener;
import org.springframework.messaging.Message;
import org.springframework.messaging.handler.annotation.Header;
import org.springframework.messaging.simp.SimpMessagingTemplate;
import org.springframework.stereotype.Component;

@Component
@EnableBinding(Sink.class) // Sink – это интерфейс, предоставляющий Binding
для входных сообщений
public class AtLeastOnceConsumer {

    @StreamListener(Sink.INPUT)
    public void handleMessage(Message<String> message, @Header(name =
"kafka_offset") String offset) {
        // Обработка сообщения
        System.out.println("Received message: " + message.getPayload());
        // После успешной обработки подтверждаем сообщение
        // Spring Cloud Stream автоматически подтвердит сообщение после
завершения метода
        // благодаря ackMode=manual и настроенному acknowledgment
    }
}
```

At most once

```
spring:
  cloud:
    stream:
      bindings:
        input:
          destination: my-topic
          group: my-consumer-group
          content-type: application/json
          consumer:
            ackMode: batch # Автоматическое подтверждение после пакета
сообщений
```

```
import org.springframework.cloud.stream.annotation.EnableBinding;
import org.springframework.cloud.stream.annotation.StreamListener;
```

```

import org.springframework.messaging.Message;
import org.springframework.stereotype.Component;

@Component
@EnableBinding(Sink.class)
public class AtMostOnceConsumer {

    @StreamListener(Sink.INPUT)
    public void handleMessage(Message<String> message) {
        // Обработка сообщения
        System.out.println("Received message: " + message.getPayload());
        // Смещение будет автоматически зафиксировано после получения
        сообщения
    }
}

```

Mostly Once

Это гибридный режим, который стремится быть чем-то средним между At least once и At most once. Он предполагает, что сообщения будут доставлены обычно один раз, но иногда, в случае сбоев, может быть обработано больше одного раза. Для реализации такого режима в Spring Cloud Stream потребуется дополнительная логика, например, фильтрация дублированных сообщений или использование уникальных идентификаторов сообщений.

В рамках Spring Cloud Stream, можно обработать Mostly Once с использованием уникальных идентификаторов сообщений или кеширования состояния, чтобы отфильтровать повторно обработанные сообщения.

```

spring:
  cloud:
    stream:
      bindings:
        input:
          destination: my-topic
          group: my-consumer-group
          content-type: application/json
          consumer:
            ackMode: manual # Ручное подтверждение
            maxAttempts: 3 # Максимальное количество попыток

```

```

import org.springframework.cloud.stream.annotation.EnableBinding;
import org.springframework.cloud.stream.annotation.StreamListener;

```

```

import org.springframework.messaging.Message;
import org.springframework.messaging.handler.annotation.Header;
import org.springframework.messaging.simp.SimpMessagingTemplate;
import org.springframework.stereotype.Component;

import java.util.HashSet;
import java.util.Set;

@Component
@EnableBinding(Sink.class)
public class MostlyOnceConsumer {

    private Set<String> processedMessageIds = new HashSet<>();

    @StreamListener(Sink.INPUT)
    public void handleMessage(Message<String> message, @Header("messageId")
String messageId) {
        if (processedMessageIds.contains(messageId)) {
            System.out.println("Duplicate message: " + messageId);
            return; // Пропускаем дублированное сообщение
        }
        // Обработка сообщения
        System.out.println("Received message: " + message.getPayload());
        // Добавляем идентификатор в обработанные
        processedMessageIds.add(messageId);
        // После успешной обработки подтверждаем сообщение вручную
        // Spring Cloud Stream подтвердит сообщение после выполнения метода
    }
}

```

Как потребители получают сообщения от брокера?

Kafka использует pull-модель для извлечения сообщений. Потребители запрашивают сообщения у брокеров, а не брокеры отправляют сообщения потребителям. Это позволяет потребителям контролировать скорость, с которой они получают сообщения.

Потребители отправляют запросы на получение данных от брокера, указывая топик, партицию и начальное смещение для каждой партиции.

Брокер отвечает сообщениями с объемом до указанного максимального предела в байтах.

Для чего нужны методы `subscribe()` и `poll()`?

Метод `subscribe()` используется для подписки на один или несколько топиков. Фактически он не извлекает никаких данных.

Метод `poll()`, с другой стороны, используется для извлечения данных из топиков. Он возвращает записи, которые были опубликованы с момента последнего запроса топиков и партиций. Метод `poll()` обычно вызывается в цикле для непрерывного получения данных.

Для чего нужен метод `position()`?

Метод `position()` возвращает смещение следующей записи, которая будет извлечена для данной партиции. Это полезно для отслеживания хода получения данных и может использоваться в сочетании с методом `committed()`, чтобы определить насколько сильно потребитель отстал от своего последнего комита оффсета. Эта информация может быть ценной для мониторинга и управления показателями потребителей.

Для чего нужен координатор группы?

Координатор группы отвечает за управление группами потребителей. Он управляет членством в группах потребителей, назначает партиции потребителям внутри группы и управляет фиксацией смещения. Когда потребитель присоединяется к группе или покидает ее, координатор группы запускает перебалансировку для переназначения партиций среди оставшихся потребителей.

Для чего нужен `Consumer heartbeat thread`?

`Consumer Heartbeat Thread` отвечает за отправку периодических сигналов брокеру Kafka (в частности, координатору группы).

Эти сигналы указывают на то, что потребитель жив и все еще является частью группы потребителей. Если потребитель не отправляет данные сигналы в течение настроенного периода, он считается неживым, и координатор группы инициирует перебалансировку для переназначения его партиций другим потребителям в группе.

Как Kafka обрабатывает сообщения?

Kafka поддерживает два основных способа обработки сообщений:

- **Queue:** каждое сообщение обрабатывается одним потребителем в группе потребителей. Это достигается за счет наличия в группе нескольких потребителей, каждый из которых считывает данные из отдельных партиций.

- **Publish-Subscribe**: все сообщения обрабатываются всеми потребителями. Это достигается за счет того, что каждый потребитель находится в своей собственной группе потребителей, что позволяет всем потребителям читать все сообщения.

Как Kafka обрабатывает задержку консьюмера?

Задержка (лаг) консьюмера в Kafka относится к разнице между оффсетом последнего созданного сообщения и оффсетом последнего полученного сообщения. Kafka предоставляет инструменты и API для мониторинга задержек консьюмеров такие, как инструмент командной строки `Kafka Consumer Groups` и `API AdminClient`. Высокая задержка консьюмеров может указывать на проблемы с производительностью или недостаточную пропускную способность консьюмеров. Kafka не обрабатывает задержки автоматически, но предоставляет информацию, необходимую приложениям для принятия решений о масштабировании или оптимизации производительности.

Сколько консьюмеров имеет смысл держать в группе?

Не больше числа партиций: каждую партицию в группе читает ровно один консьюмер, лишние простаивают (как резерв на случай падения другого).

Чем `eager rebalance` отличается от `cooperative`?

`Eager`: все консьюмеры отдают все партиции, группа на время стоит. `Cooperative` (`CooperativeStickyAssignor`): переносится только нужная часть партиций, остальные читают дальше; может занять несколько итераций. `Static membership` (`group.instance.id`) позволяет перезапустить консьюмер без `rebalance` в пределах `session.timeout.ms`. Новый протокол `KIP-848` (`group.protocol=consumer`, Kafka 4.0) делает `rebalance` инкрементальным всегда.

Как корректно остановить `KafkaConsumer`?

`Shutdown hook` вызывает `wakeup()` (единственный потокобезопасный метод) → `poll()` бросает `WakeupException` → поток консьюмера вызывает `close()`: коммит `offsets + LeaveGroup`, `rebalance` сразу, без ожидания `session.timeout.ms`. Hook ждёт `mainThread.join()`.

Offsets и гарантии доставки

Для чего нужны методы `commitSync()` и `commitAsync()`?

Эти методы используются для фиксации смещений:

- **`commitSync()`**: синхронно фиксирует последнее смещение, возвращенное `poll()`. Он будет повторять попытку до тех пор, пока не завершится успешно или не столкнется с непроверяемой ошибкой.
- **`commitAsync()`**: асинхронно фиксирует смещения. Он не повторяет попытку при сбое, что делает его более быстрым, но менее надежным, чем `commitSync()`. Выбор между этими методами зависит от баланса между производительностью и надежностью, требуемого приложением.

Где хранятся `offsets` и что коммитится?

Во внутреннем compacted-топике `__consumer_offsets` (50 партиций, RF 3), ключ — «группа + топик + партиция». Коммитится offset **следующего** сообщения для чтения. Брокер-лидер партиции `hash(group.id) % 50` — group coordinator этой группы.

Почему авто-коммит может потерять сообщения?

Он коммитит внутри `poll()` offsets записей **предыдущего** `poll()`. Если обработка асинхронная и не закончилась до следующего `poll()`, необработанное будет закоммичено и при падении потеряется.

Как получить `exactly once`?

Kafka → Kafka: Transactional API (`transactional.id`, `sendOffsetsToTransaction`, консьюмеры с `isolation.level=read_committed`) или `processing.guarantee=exactly_once_v2` в Kafka Streams. Kafka → внешняя система: at least once + идемпотентная запись (уникальный ID / upsert) или offset в той же транзакции БД.

API и экосистема

Основные API Kafka

- Producer API
- Consumer API
- Streams API
- Connector API

Какова роль Producer API?

Используется для публикации потока сообщений в топики Kafka. Он управляет партиционированием сообщений, сжатием и балансировкой нагрузки между несколькими брокерами. Продюсер также отвечает за повторные неудачные попытки публикации и может быть настроен на различные уровни гарантий доставки.

Какова роль Consumer API?

Обеспечивает механизм для потребления сообщений топиков. Оно позволяет приложениям и микросервисам читать данные, поступающие в Kafka, и обрабатывать их для дальнейшего использования, будь то хранение, анализ или реактивная обработка.

Какова роль Connector API?

Connector API в Apache Kafka является частью Kafka Connect, которая представляет собой инфраструктуру для интеграции внешних систем с Kafka. Connector API играет ключевую роль в упрощении процесса подключения различных источников данных и систем-приемников к Kafka, предоставляя возможность автоматического перемещения данных между ними.

Какова роль Streams API?

Это компонент Apache Kafka, предназначенный для создания приложений и микросервисов, которые обрабатывают потоки данных в реальном времени. Его основная роль заключается в том, чтобы позволить разработчикам легко обрабатывать и анализировать данные, поступающие в виде непрерывных потоков из топиков. Kafka Streams API предоставляет высокоуровневый интерфейс для выполнения таких операций, как фильтрация, агрегация, объединение данных и вычисление оконных функций.

Какова роль Transactions API?

Kafka Transactions API позволяет выполнять атомарные обновления для нескольких топиков. Он включает exactly-once гарантию для приложений, которые читают данные из одного топика и пишут в другой. Это особенно полезно для приложений потоковой обработки, которым необходимо гарантировать, что каждое входное событие влияет на выходные данные ровно один раз, даже в случае сбоев.

Какова роль Quota API?

Quota API позволяет настраивать квоты для каждого клиента для ограничения скорости создания или потребления данных, чтобы один клиент не потреблял слишком много ресурсов брокера. Это помогает обеспечить справедливое распределение ресурсов и предотвратить сценарии отказа в обслуживании.

Какова роль AdminClient API?

AdminClient API предоставляет операции для управления топиками, брокерами, конфигурацией и другими объектами Kafka. Его можно использовать для создания, удаления и описания топиков, управления списками ACL, получения информации о кластере и программного выполнения других административных задач.

Для чего нужен Streams DSL?

Kafka Streams DSL предоставляет высокоуровневый API для операций потоковой обработки. Он позволяет разработчикам описывать сложную логику обработки, такую как фильтрация, преобразование, агрегирование и объединение потоков данных. DSL абстрагирует многие низкоуровневые детали потоковой обработки, упрощая создание и обслуживание приложений потоковой обработки.

В чем разница между Kafka Consumer и Kafka Stream?

Kafka Consumer - это клиент, который читает данные из топика и производит некоторую обработку. Обычно используется для простых сценариев получения данных. **Kafka Stream**, с другой стороны, более продвинутый клиент, который может потреблять, обрабатывать и класть данные обратно в Kafka. Он предоставляет DSL для сложных операций потоковой обработки, таких как фильтрация, преобразование, агрегирование и объединение потоков.

В чем разница между Kafka Streams и Apache Flink?

Kafka Streams и Apache Flink — это два мощных инструмента для обработки потоков данных в режиме реального времени, но они различаются по архитектуре, возможностям и сценариям применения.

Сравнение Kafka Streams и Apache Flink

Критерий	Kafka Streams	Apache Flink
Архитектура	Встроенная библиотека, работающая внутри приложения. Зависит от Kafka.	Независимая распределенная система потоковой обработки данных с возможностью интеграции с различными источниками и приемниками данных.
Обработка данных	Обрабатывает потоки событий непосредственно из Kafka. Подходит для обработки событий и транзакционных данных с минимальной задержкой.	Поддерживает как потоковую (streaming), так и пакетную (batch) обработку данных. Специализируется на сложной обработке событий с гибкими возможностями управления состоянием.
Зависимость от Kafka	Построена исключительно вокруг Kafka. Требует Kafka для получения и отправки данных.	Работает с широким спектром источников данных (Kafka, HDFS, базы данных и т. д.). Kafka — лишь один из многих источников.
Установка	Легко интегрируется в существующее Java/Scala-приложение как библиотека. Не требует развертывания кластеров.	Требует отдельного кластера для выполнения, что подходит для высокопроизводительных распределенных систем.
Управление состоянием	Встроенное состояние с использованием RocksDB, также поддержка репликации состояния.	Имеет развитую систему управления состоянием, поддерживает сложные функции восстановления состояния и обработки данных.
Гарантия доставки	Поддерживает "at-least-once" и "exactly-once" семантику, когда Kafka настроена соответствующим образом.	Имеет гибкие гарантии доставки: поддержка "exactly-once", "at-least-once" и "at-most-once".
Масштабируемость	Масштабируется автоматически вместе с Kafka-партициями. Каждая инстанция потребителя Kafka обрабатывает свою партицию.	Поддерживает масштабирование на уровне задач (task), с более гибкой моделью масштабирования и управления ресурсами.

Критерий	Kafka Streams	Apache Flink
Обработка событий	Подходит для обработки событий с низкой задержкой и транзакционными требованиями.	Специализируется на сложной обработке событий, таких как windowing, агрегирование и работа с изменяющимся состоянием. Поддерживает сложные аналитические операции.
Инструменты и API	Легковесная библиотека с простыми API для работы с потоками данных. Основные операции — фильтрация, маппинг, объединение потоков, windowing.	Продвинутая система с богатыми API для сложных вычислений, поддерживающая потоковую и пакетную обработку, обработку событий и контроль сложных бизнес-процессов.
Требования к ресурсам	Менее ресурсоемка, так как не требует отдельного кластера. Работает в рамках JVM-приложения.	Требует более высоких вычислительных ресурсов, так как выполняется на отдельном кластере и поддерживает высокую степень параллелизма.

Когда выбрать Kafka Streams

- Если вы уже используете Kafka и вам нужна легковесная библиотека для обработки данных непосредственно внутри вашего приложения.
- Для сценариев с низкой задержкой, где данные приходят из Kafka и должны быть быстро обработаны с минимальными накладными расходами.
- Если вам нужно встроить обработку потоков данных в существующую Java/Scala программу без необходимости развертывания отдельных кластеров.

Когда выбрать Apache Flink

- Если вы работаете с потоковой и пакетной обработкой данных, где источники и приемники могут быть не только Kafka, но и другие системы (например, HDFS, базы данных).
- Для сложных задач обработки событий, требующих управления состоянием, временных окон, аналитики и восстановления после сбоев.
- Если ваш проект требует высокой производительности, гибкости, точных гарантий доставки и распределенной обработки в кластере.

Заключение

- **Kafka Streams** — это идеальный выбор, если ваша инфраструктура уже основана на Kafka, и вам нужна быстрая и легковесная обработка потоков данных.
- **Apache Flink** — это мощный инструмент для сложных аналитических задач, потоковой обработки данных в режиме реального времени с поддержкой сложных схем обработки, который предоставляет больше возможностей для работы с разнообразными источниками данных.

Для чего нужна Schema Registry?

Kafka Schema Registry предоставляет RESTful интерфейс для хранения и извлечения схем Avro. Schema Registry используется совместно с Kafka для обеспечения совместимости схем данных между производителями и потребителями. Это особенно полезно при разработке моделей данных с течением времени, сохраняя обратную и прямую совместимость.

Как Kafka обеспечивает версионирование сообщений?

Сама по себе Kafka не обеспечивает версионирование сообщений напрямую, но предоставляет механизмы, позволяющие реализовывать управление версиями. Одним из распространенных подходов является включение поля версии в схему сообщения. Для более сложных задач управления версиями используются реестры схем (например, Confluent Schema Registry), которые могут управлять изменением схемы и совместимостью.

Для чего нужен Kafka Mirror Maker?

Это инструмент, позволяющий реплицировать данные между кластерами Kafka, потенциально находящихся в разных дата-центрах. Он работает, потребляя данные из одного кластера и передавая в другой. Можно использовать для создания резервной копии данных, объединения данных из нескольких дата-центров в единое хранилище или для переноса данных между кластерами.

Сравнение с другими системами

В чем разница между Kafka и RabbitMQ?

RabbitMQ и **Apache Kafka** — это две популярные системы обмена сообщениями, каждая из которых имеет свои особенности и используется для разных типов приложений. Вот основные различия между ними:

1. Архитектура

- **RabbitMQ** использует **очереди сообщений**. Сообщения отправляются в очередь, и один потребитель извлекает сообщение из очереди для обработки.
- **Apache Kafka** использует **топики и партиции**. Сообщения отправляются в топики, которые могут быть разделены на партиции, и несколько потребителей могут читать эти сообщения в любом порядке. Kafka ориентирован на большие потоки данных и масштабируемость.

2. Модель доставки сообщений

- **RabbitMQ**: Сообщения передаются в очереди, и каждый потребитель получает одно сообщение. Сообщения могут быть подтверждены (acknowledged) или отклонены (rejected). RabbitMQ гарантирует, что сообщение будет доставлено хотя бы одному потребителю.
- **Kafka**: Сообщения сохраняются в топиках на длительный срок, и потребители могут читать их в любом порядке. Kafka гарантирует доставку сообщений всем потребителям, если они подписаны на топик, и может позволить многократное чтение старых сообщений.

3. Гарантии доставки

- **RabbitMQ**: Предоставляет подтверждения доставки и может повторно отправить сообщение, если потребитель не подтвердил его получение. Можно настроить разные уровни надежности (например, за счет использования подтверждений или транзакций).
- **Kafka**: Сообщения сохраняются на диске, что позволяет потребителям считывать их в любое время. Kafka гарантирует доставку сообщений при определенной конфигурации репликации и сохранения.

4. Производительность и масштабируемость

- **RabbitMQ**: Лучше подходит для небольших и средних систем, где требуется высокая надежность и гарантированная доставка. Он поддерживает **горизонтальное масштабирование**, но требует дополнительных усилий для настройки и управления.
- **Kafka**: Отличается высокой **производительностью** и возможностью обработки больших объемов данных. Kafka легко масштабируется за счет **партиционирования** и репликации данных.

5. Потребительская модель

- **RabbitMQ:** Один потребитель получает одно сообщение. Если потребитель не успевает обработать сообщение, оно может быть повторно отправлено.
- **Kafka:** Потребители могут читать сообщения независимо друг от друга. Kafka сохраняет все сообщения в топиках, и потребители могут читать их в любое время. Kafka также поддерживает концепцию **групп потребителей**, где каждый потребитель группы обрабатывает разные партиции.

6. Использование и кейсы

- **RabbitMQ:** Идеален для обработки запросов и ответов, распределенных приложений, микросервисов с гарантией доставки, бизнес-процессов с очередями задач.
- **Kafka:** Используется для обработки потоков данных, интеграции с большими данными, записи журналов, мониторинга, обработки событий в реальном времени и сохранения больших объемов данных для последующего анализа.

7. Производители и потребители

- **RabbitMQ:** Один производитель отправляет сообщения в очередь, и несколько потребителей могут обрабатывать эти сообщения.
- **Kafka:** Множество производителей могут отправлять сообщения в топик, и несколько потребителей могут читать их одновременно, поддерживая масштабируемость.

8. Сообщения и хранение

- **RabbitMQ:** Сообщения удаляются из очереди после их обработки потребителем. Хранение сообщений обычно краткосрочное.
- **Kafka:** Сообщения сохраняются на диске в топиках до тех пор, пока не истечет срок хранения (по конфигурации). Это позволяет повторно читать данные.

В чем разница между Kafka и Flume?

Apache Kafka и **Apache Flume** — это два популярных инструмента для обработки и передачи данных, однако они имеют разные цели и архитектуры. Вот основные различия между ними:

1. Назначение и использование

- **Kafka:** Это распределенная платформа для потоковой передачи данных, которая обеспечивает высокую пропускную способность и низкую задержку для обработки больших объемов данных. Kafka используется для создания стриминговых приложений и обработки данных в реальном времени. Она может быть использована для передачи логов, событий, метрик и других данных, требующих высокой доступности и масштабируемости.
- **Flume:** Это распределенная система для сбора, агрегации и передачи логов и событий. Flume обычно используется для доставки логов с серверов в HDFS, HBase или другие системы хранения данных. Его основное предназначение — это сбор данных из различных источников (например, лог-файлов) и передача их в системы хранения или аналитики.

2. Архитектура

- **Kafka:** В Kafka данные отправляются в топики и партиции, которые могут быть независимо прочитаны несколькими потребителями. Kafka ориентирована на высокую пропускную способность и масштабируемость. Это решает задачу обработки потоковых данных и событий в реальном времени.
- **Flume:** Flume состоит из **источников (sources)**, **каналов (channels)** и **приемников (sinks)**. Источник получает данные, канал их буферизует, а приемник отправляет их в конечную систему. Flume использует систему "event-based" и часто применяется для сбора логов.

3. Хранение данных

- **Kafka:** Kafka сохраняет сообщения на диске в течение длительного времени (по умолчанию — до 7 дней) в топиках. Потребители могут читать данные в любой момент времени, и Kafka поддерживает концепцию **сохранения и ретрансляции данных**.
- **Flume:** Flume не имеет встроенного механизма долговременного хранения. Он просто передает данные в назначенные места хранения (например, HDFS). Данные в Flume не сохраняются долго, и если система хранения не доступна, они теряются.

4. Производительность

- **Kafka:** Kafka предназначен для работы с высокими объемами данных. Он поддерживает масштабируемость как по производителям,

так и по потребителям, и может обрабатывать миллионы сообщений в секунду с минимальной задержкой.

- **Flume:** Flume может быть менее масштабируемым по сравнению с Kafka и больше ориентирован на сбор логов и событий с различных источников. Хотя Flume тоже может обрабатывать большие объемы данных, он не предназначен для работы с такими большими потоками, как Kafka.

5. Использование и кейсы

- **Kafka:** Используется для стриминга данных, аналитики в реальном времени, интеграции различных систем, работы с большими данными и построения событийных приложений.
- **Flume:** Используется для сбора, агрегации и передачи логов и событий в системы хранения, такие как HDFS, HBase, или внешние системы. Это идеальный выбор для организации потоков логирования и мониторинга.

6. Поддержка и интеграция

- **Kafka:** Kafka поддерживает широкий спектр интеграций и может быть использован с различными системами для построения распределенных приложений и аналитических решений.
- **Flume:** Flume ориентирован на интеграцию с Hadoop-экосистемой, и основное его использование — это интеграция с HDFS, HBase и другими хранилищами данных в этой экосистеме.

7. Потребительская модель

- **Kafka:** Kafka поддерживает много потребителей, которые могут читать из одного и того же топика независимо, а также возможность **повторного прочтения данных**.
- **Flume:** Flume имеет фиксированную схему доставки данных и не поддерживает такую гибкость, как Kafka в части потребителей и обработки.

8. Гарантии доставки

- **Kafka:** Kafka поддерживает **гарантии доставки** с различными уровнями подтверждения (acknowledgment), а также может обеспечивать **доставку сообщений точно один раз** (exactly-once semantics).
- **Flume:** Flume обеспечивает базовые гарантии доставки, но они менее строгие, чем у Kafka, и больше ориентированы на

устойчивость к сбоям, а не на гарантированную доставку.